Stochastic Particle Flow and Differentiable PF

Amresh Verma

April 26, 2026

Abstract

We present implementations and empirical evaluations across four interconnected problems in sequential Monte Carlo: pseudo-time scheduling for stochastic particle flow, gradient quality in differentiable particle filters, Hamiltonian Monte Carlo with invertible flows, and neural acceleration of optimal transport resampling. On the bearing-only localisation benchmark, solving the BVP-optimal pseudo-time schedule offline reduces the flow stiffness objective by 4.2%, and RMSE by 16.2%. Among four differentiable gradient estimators, OT-based resampling achieves the best signal-to-noise ratio (0.43 vs. 0.08 for straight-through), while remaining computationally competitive. Coupling LEDH particle flow to HMC raises acceptance from 14% (PMMH) to 58%, but ESS per second still favours PMMH by two orders of magnitude because the ODE integration dominates runtime; an L-HNN gradient surrogate recovers a 2.3× wall-clock speedup at the cost of increased parameter estimation bias. A neural Sinkhorn surrogate raises filter ESS from 4.6 to 8.6 and is shown to require only (θ, t, y_t) as context to generalise without re-training; Hyper-DeepONet closes the expressiveness gap of standard DeepONet but suffers from gradient norm explosion at long horizons. On a cross-dynamics neural state-space benchmark, DPF-HMC achieves the best state-estimation RMSE on three of four dynamics and the most accurate process-noise recovery, while Particle Gibbs wins on test log-likelihood due to a systematic R-calibration asymmetry. Across all experiments the Markovian adapter approximation emerges as the binding constraint; we identify a recurrent particle filter as the highest-priority architectural fix.

Contents

1 Introduction **3**

2 Theoretical Frameworks **3**

2.1 Dai–Daum Stochastic Particle Flow Filter 3

2.2 PF-PF with Dai–Daum Optimized Particle Flow 4

2.3 Differentiable Particle Filtering 5

2.3.1 Soft Resampling 6

2.3.2 Entropy-Regularized Optimal Transport Resampling 6

2.3.3 Stop-Gradient Baseline 7

2.3.4 Particle Transformer 7

2.4 Particle Marginal Metropolis-Hastings (PMMH) . . . 8
2.5 Differentiable LEDH Particle Filter . . . 9
2.6 HMC with Differentiable Particle Filter Target . . . 10
2.7 L-HNN Accelerated HMC . . . 11
2.7.1 Architecture . . . 11
2.7.2 Training . . . 11
2.7.3 Synchronized Leapfrog . . . 11
2.7.4 Online Error Monitoring . . . 12
2.7.5 Computational Savings . . . 12
2.8 Neural Acceleration of OT Resampling . . . 12
2.8.1 Theoretical Motivation . . . 13
2.8.2 Monotone Gradient Network (mGradNet-M) . . . 13
2.8.3 DeepONet Formulation . . . 14
2.8.4 Hyper-DeepONet . . . 14
2.8.5 Training . . . 15
2.9 State-Space LSTM . . . 15
2.9.1 Gaussian State-Space LSTM Model . . . 16
2.9.2 Particle Gibbs with Optimal Proposal . . . 16
2.9.3 DPF-HMC Pipeline . . . 17
2.9.4 PMMH Baseline . . . 18
2.9.5 Method Comparison . . . 18

3 Results and Discussions **19**
3.1 Question 1 . . . 19
3.1.1 Answer (a) : Replication of Dai(22) . . . 19
3.1.2 Answer (b) : PFPF_Dai Vs. PFPF_LEDH . . . 21
3.2 Question 2 . . . 22
3.2.1 Answer (i) : Bias-Var-Speed trade off . . . 23
3.2.2 Answer (ii) : DPF Comparison . . . 24
3.3 Bonus Question 1 . . . 27
3.3.1 Answer (a) : Estimation with PF-PF . . . 27
3.3.2 Answer (b) : HMC Vs. PMMH . . . 28
3.3.3 Answer (c) : Challenges . . . 30
3.4 Bonus Question 2 . . . 32
3.4.1 Answer (a) : Chaudhari(25) . . . 32
3.4.2 Answer (b) : Jha(25) . . . 35
3.5 Bonus Question 3 . . . 37
3.5.1 Answer (a) : DPF-HMC Vs. PG . . . 37
3.5.2 Answer (b) : Pipeline, Limitations, and Future . . . 39

4 Conclusion **43**

References **44**

1 Introduction

This work builds on the first part of the study, where classical filtering methods and deterministic particle flow approaches were developed and analyzed for nonlinear, non-Gaussian state-space models. In particular, while particle flow methods alleviate some of the degeneracy and proposal mismatch issues observed in standard particle filters, their deterministic formulations can suffer from limited flexibility and reduced robustness in highly complex or high-dimensional settings [1, 2].

Motivated by these challenges, this work focuses on stochastic particle flow methods [3] and differentiable particle filters (DPFs) [4] as a unified framework for scalable and learnable sequential inference. Stochastic particle flows extend deterministic formulations by introducing controlled diffusion, improving robustness and exploration in complex posterior landscapes. Differentiable particle filters, in turn, enable gradient-based optimization of the filtering pipeline by parameterizing proposal distributions, transition dynamics, and resampling mechanisms. In particular, we implement differentiable resampling schemes based on entropy-regularized optimal transport, allowing gradients to propagate through the resampling step while maintaining a balance between bias, variance, and computational cost.

Building on these components, we further explore several advanced extensions. First, we investigate the integration of invertible particle flow proposals with differentiable resampling to enable gradient-based Bayesian inference via Hamiltonian Monte Carlo (HMC) [5], addressing a key limitation of traditional particle filtering methods. Second, we examine neural approaches [6] to accelerate optimal transport resampling, reducing the computational overhead of Sinkhorn iterations. Third, we study particle-flow-based inference in neural state-space models [7], comparing differentiable particle filtering with particle Gibbs methods in terms of accuracy and efficiency.

Across all experiments, we provide a systematic empirical evaluation of the proposed methods using metrics that capture estimation accuracy, numerical stability, computational efficiency, and gradient behavior. These include standard filtering diagnostics as well as measures tailored to differentiable inference. Our results highlight the trade-offs between expressivity, stability, and efficiency, and demonstrate that transport-based and differentiable approaches offer a promising path toward scalable sequential inference in complex, high-dimensional systems.

2 Theoretical Frameworks

2.1 Dai–Daum Stochastic Particle Flow Filter

We consider nonlinear Bayesian filtering for a state-space model with prior density $p_0(x)$ and measurement likelihood $p(z \mid x)$ [3]. Rather than forming importance weights, the filter transports particles from the prior to the posterior through an artificial homotopy variable $\lambda \in [0, 1]$. The intermediate target density is defined by

$$\log \pi(x, \lambda) = (1 - \beta(\lambda)) \log p_0(x) + \beta(\lambda) \log p(z \mid x), \tag{1}$$

where $\beta(\lambda)$ is a monotone schedule with $\beta(0) = 0$ and $\beta(1) = 1$. At $\lambda = 0$, the density is the prior; at $\lambda = 1$, it becomes the posterior.

The particle flow is governed by a stochastic differential equation in λ:

$$dx_\lambda = f(x_\lambda, \lambda)\, d\lambda + L_Q\, dW_\lambda, \tag{2}$$

where $Q = L_Q L_Q^\top$ is the process-noise covariance and W_λ is a standard Wiener process. The drift is given by

$$f(x, \lambda) = K_1 \nabla_x \log \pi(x, \lambda) + K_2 \nabla_x \log p(z \mid x), \tag{3}$$

with

$$K_2 = -\dot{\beta}(\lambda)\, H^{-1}, \qquad K_1 = \frac{1}{2} Q - \frac{\dot{\beta}(\lambda)}{2} H^{-1} H_h H^{-1}. \tag{4}$$

Here H is the Hessian of the homotopy log-density and H_h is the likelihood Hessian:

$$H = (1-\beta) H_0 + \beta H_h, \qquad H_0 = \nabla_x^2 \log p_0(x), \qquad H_h = \nabla_x^2 \log p(z \mid x). \tag{5}$$

In practice, H is symmetrized and regularized numerically to ensure stable matrix solves.

To reduce stiffness, the schedule $\beta(\lambda)$ is optionally optimized by solving a one-dimensional boundary value problem. The objective penalizes rapid changes in the schedule and large curvature in the homotopy density:

$$J(\beta) = \int_0^1 \left[\frac{1}{2} \dot{\beta}(\lambda)^2 + \mu\, \kappa^*(M(\lambda)) \right] d\lambda, \qquad M(\lambda) = -\nabla_x^2 \log \pi(x, \lambda), \tag{6}$$

where $\kappa^*(M)$ is a surrogate condition number and $\mu > 0$ controls the trade-off between smoothness and stiffness reduction. The resulting optimal schedule $\beta^*(\lambda)$ is computed on a coarse mesh, then interpolated onto the finer Euler–Maruyama grid used for particle propagation.

Algorithmically, particles are first sampled from the prior, then propagated over a grid of λ values using Euler–Maruyama:

$$x_{k+1}^{(i)} = x_k^{(i)} + f\left(x_k^{(i)}, \beta_k, \dot{\beta}_k\right) \Delta\lambda + L_Q \Delta W_k^{(i)}. \tag{7}$$

After all λ steps, the posterior estimate is obtained from the propagated particles:

$$\hat{x} = \frac{1}{N} \sum_{i=1}^{N} x^{(i)}, \qquad \hat{P} = \frac{1}{N-1} \sum_{i=1}^{N} (x^{(i)} - \hat{x})(x^{(i)} - \hat{x})^\top. \tag{8}$$

This implementation follows the Dai–Daum stochastic particle flow framework [8] while adding practical numerical stabilizers: Hessian regularization, drift clipping, batched propagation, and optional schedule optimization via continuation and Adam-based minimization.

2.2 PF-PF with Dai–Daum Optimized Particle Flow

In this variant, the particle filter proposal is constructed by transporting particles through a deterministic particle flow [9], while the homotopy schedule is chosen using the stiffness-mitigated objective of Dai–Daum. At each measurement update, particles are first propagated by a standard predictive model (e.g. EKF/UKF prediction or the state transition

model), yielding a prior particle set $\{x_0^{(i)}\}_{i=1}^N$. These particles are then moved along an artificial homotopy variable $\lambda \in [0,1]$ toward the posterior.

The intermediate density is defined by the same homotopy relation, Eq (1). Rather than using the linear schedule $\beta(\lambda) = \lambda$, we compute an optimized schedule $\beta^*(\lambda)$ by minimizing the Dai–Daum stiffness objective Eq (6) where $\kappa^*(\cdot)$ is a surrogate condition number and $\mu > 0$ balances smoothness against stiffness reduction. In practice, $\beta^*(\lambda)$ is solved on a coarse mesh and interpolated onto the finer Euler–Maruyama grid used for propagation.

Given $\beta(\lambda)$ and $\dot{\beta}(\lambda)$, each particle undergoes a *deterministic* flow (Li and Coates' approach), unlike the stochastic case mentioned earlier.

$$\frac{dx}{d\lambda} = f(x, \lambda), \tag{9}$$

with drift of the form Eq. (3). Although the stochastic diffusion term is omitted, the covariance Q is retained in K_1 as a metric that shapes the flow according to the prior uncertainty, rather than as a source of randomness. In this variant, we omit the stochastic diffusion term so that the flow defines an approximately invertible transport map, which is required by the PF-PF importance correction.

The continuous flow is discretized using Euler steps on a grid $\{\lambda_k\}_{k=0}^K$:

$$x_{k+1}^{(i)} = x_k^{(i)} + f\left(x_k^{(i)}, \beta_k, \dot{\beta}_k\right) \Delta\lambda, \qquad \Delta\lambda = \frac{1}{K}. \tag{10}$$

After all particles are transported, the proposal density correction is applied through the Jacobian determinant of the flow map T:

$$\log w^{(i)} \leftarrow \log w^{(i)} + \log p(z \mid x_K^{(i)}) - \log \left| \det \frac{\partial T}{\partial x_0} \right|. \tag{11}$$

The Jacobian correction appears with a minus sign because we accumulate the log-determinant of the forward transport map $T : x_0 \mapsto x_K$; equivalently, the proposal density contains the inverse Jacobian factor $|\det(\partial T/\partial x_0)|^{-1}$. Since the flow is implemented numerically, the log-determinant is approximated by integrating the divergence of the vector field:

$$\log \left| \det \frac{\partial T}{\partial x_0} \right| \approx \sum_{k=0}^{K-1} \operatorname{tr}\left(\frac{\partial f}{\partial x}(x_k, \lambda_k) \right) \Delta\lambda. \tag{12}$$

The weights are then normalized, and systematic resampling is applied when the effective sample size falls below a threshold.

This yields a practical particle-flow particle filter in which the proposal is shaped by a Dai–Daum optimized homotopy while the posterior correction follows the Li–Coates invertible-flow framework.

2.3 Differentiable Particle Filtering

Standard particle filters use multinomial or systematic resampling to combat weight degeneracy, but the discrete index-selection step is not differentiable: gradients cannot flow

backward through the resampling operation. This prevents using gradient-based methods (e.g. backpropagation, HMC) to learn model parameters or tune the filter end-to-end. Several approaches have been proposed to make resampling differentiable; we implement and compare three: soft resampling, entropy-regularized optimal transport (OT) resampling, and a learned Particle Transformer.

All variants share a common bootstrap particle filter structure. Given observations $y_{1:T}$, particles $\{x_t^{(i)}\}_{i=1}^N$ are propagated through:

$$x_t^{(i)} \sim f(x_t \mid x_{t-1}^{(i)}), \qquad \log w_t^{(i)} \leftarrow \log w_t^{(i)} + \log g(y_t \mid x_t^{(i)}). \tag{13}$$

After weight normalization, the incremental log-likelihood is accumulated as

$$\log \hat{p}(y_{1:T}) = \sum_{t=1}^{T} \log \left(\frac{1}{N} \sum_{i=1}^{N} \tilde{w}_t^{(i)} \right), \tag{14}$$

where $\tilde{w}_t^{(i)}$ are unnormalized weights. When the effective sample size $\mathrm{ESS} = 1/\sum_i (w_t^{(i)})^2$ falls below a threshold τN, the particles are resampled. The choice of resampling scheme distinguishes the four filter variants described below.

2.3.1 Soft Resampling

Following [10], ancestor indices are sampled not from the normalized weights w directly but from a mixture distribution:

$$q_i = (1-\alpha)\, w_i + \frac{\alpha}{N}, \tag{15}$$

where $\alpha \in [0,1]$ interpolates between hard resampling ($\alpha = 0$) and uniform sampling ($\alpha = 1$). Because $q_i > 0$ for all i, the log-probability $\log q_i$ is always well-defined and gradients with respect to the original weights w can flow through q. To maintain an unbiased estimator, the resampled weights are corrected:

$$\tilde{w}_{a_i} = \frac{w_{a_i}}{q_{a_i}}, \tag{16}$$

where a_i is the sampled ancestor index. This fulfils requirement (a) of the assignment: enabling backpropagation through weights via a simple mixture with uniform.

2.3.2 Entropy-Regularized Optimal Transport Resampling

The Differentiable Ensemble Transform (DET) of [11] replaces discrete resampling with a smooth transport map. The source measure is the weighted empirical distribution $\alpha = \sum_i w_i\, \delta_{x_i}$ and the target is the uniform measure $\beta = \frac{1}{N}\sum_j \delta_{x_j}$ on the same particle support. The entropy-regularized optimal transport plan is defined as

$$P^{\varepsilon} = \arg\min_{P \in \mathcal{U}(w, \mathbf{1}/N)} \sum_{i,j} P_{ij}\, C_{ij} \; - \; \varepsilon\, H(P), \tag{17}$$

where $C_{ij} = \frac{1}{2}\|x_i - x_j\|^2$ is the half squared Euclidean cost, $H(P) = -\sum_{ij} P_{ij} \log P_{ij}$ is the entropic regularizer, and $\mathcal{U}(w, \mathbf{1}/N)$ denotes the set of transport plans whose row marginals equal w and column marginals equal $\mathbf{1}/N$.

The solution is computed via Sinkhorn's algorithm in dual form, following Feydy et al. (2019). Dual potentials (f, g) are initialized to zero and updated by damped fixed-point iterations:

$$f_i \leftarrow \tfrac{1}{2}\left(f_i - \varepsilon \log \sum_j b_j \exp\left(\frac{g_j - C_{ij}}{\varepsilon}\right)\right), \tag{18}$$

$$g_j \leftarrow \tfrac{1}{2}\left(g_j - \varepsilon \log \sum_i a_i \exp\left(\frac{f_i - C_{ij}}{\varepsilon}\right)\right), \tag{19}$$

where all log-sum-exp operations are implemented with the standard numerical stability trick of subtracting the maximum before exponentiation. The transport matrix is recovered as

$$P_{ij} = a_i\, b_j \exp\left(\frac{f_i + g_j - C_{ij}}{\varepsilon}\right). \tag{20}$$

Resampled particles are then obtained via barycentric projection:

$$\tilde{x}_j = N \sum_{i=1}^{N} P_{ij}\, x_i, \tag{21}$$

with uniform output weights $\tilde{w}_j = 1/N$. The entire operation is differentiable because Sinkhorn iterations involve only smooth operations (log-sum-exp, matrix products, elementwise exponentials).

The regularization parameter ε controls a bias–variance–speed trade-off. Small ε recovers hard resampling (low bias, high gradient variance, more Sinkhorn iterations needed for convergence), while large ε produces smoother but more biased transport plans that converge in fewer iterations. In our experiments, we tune ε and the number of Sinkhorn iterations to balance these factors.

2.3.3 Stop-Gradient Baseline

As an ablation, we implement a stop-gradient particle filter following [12]. Standard multinomial resampling is applied, but the resampled particles and weights are wrapped with a gradient stop:

$$\tilde{x} = \mathrm{stopgrad}(x_{\mathrm{resampled}}), \qquad \tilde{w} = \mathrm{stopgrad}(w_{\mathrm{resampled}}). \tag{22}$$

This blocks gradient flow through the resampling step while preserving gradients through dynamics and observation models within each time step. Comparing against this baseline isolates the specific contribution of differentiable resampling (OT or soft) from the contribution of having differentiable dynamics and observation models.

2.3.4 Particle Transformer

Following [13], we implement a learned resampling network based on the Set Transformer architecture [14]. The Particle Transformer maps N weighted particles $\{(x_i, w_i)\}_{i=1}^{N}$ to N equally-weighted output particles using weighted multi-head self-attention:

$$\mathrm{WeightedAttn}(Q, K, V, w) = \mathrm{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + \log w\right) V, \tag{23}$$

where the particle weights w_i modulate the attention logits additively in log-space, ensuring that high-weight particles receive proportionally more attention.

The architecture consists of an encoder and a decoder. The encoder applies an input projection from state dimension d to model dimension d_{model}, followed by two weighted self-attention blocks. The decoder begins with a learned seed bank of fixed size S, which is interpolated to N seeds via bilinear interpolation to handle variable particle counts. Two rounds of self-attention followed by weighted cross-attention against the encoder output produce the decoder representation, which is projected back to state dimension d by a final linear layer. Scale equivariance is enforced by normalizing inputs to $[-1, 1]$ per dimension before the encoder and inverting the normalization after the decoder.

Training proceeds in two stages. In the first stage (individual pre-training), a standard particle filter is run for multiple episodes to collect pre-resampling particle sets $\{(x^{(b)}, w^{(b)})\}_{b=1}^{B}$. The Particle Transformer is trained to minimize a KDE-based KL divergence loss:

$$\mathcal{L}_{\text{KL}} = -\sum_{i=1}^{N} w_i \log \hat{q}(x_i), \tag{24}$$

where $\hat{q}$ is a Gaussian kernel density estimate over the resampled particles with bandwidth h:

$$\hat{q}(x) = \frac{1}{N} \sum_{j=1}^{N} \frac{1}{(2\pi h^2)^{d/2}} \exp\left(-\frac{\|x - \tilde{x}_j\|^2}{2h^2} \right). \tag{25}$$

In the second stage (end-to-end training), the pre-trained Particle Transformer is inserted into the differentiable filter, and all parameters (network weights and optionally SSM parameters) are jointly optimized by maximizing the filter log-likelihood $\log \hat{p}(y_{1:T})$ from Eq. (14). Following the recommendation of [13], gradients are stopped across time steps to avoid the exploding-gradient problem, and global gradient clipping is applied.

2.4 Particle Marginal Metropolis-Hastings (PMMH)

PMMH [15] uses the pseudo-marginal principle: the intractable marginal likelihood is replaced by an unbiased particle filter estimate, and the resulting noisy acceptance ratio still targets the correct posterior. The algorithm proceeds as a standard Metropolis-Hastings chain over θ:

1. Propose $\theta' \sim q(\theta' \mid \theta)$ using a Gaussian random walk with step size σ_{RW}.
2. Run a bootstrap particle filter with N particles under θ' to obtain $\log \hat{p}(y_{1:T} \mid \theta')$.
3. Accept with probability

$$\alpha = \min\left(1, \; \frac{\hat{p}(y_{1:T} \mid \theta')\, p(\theta')}{\hat{p}(y_{1:T} \mid \theta)\, p(\theta)} \right). \tag{26}$$

The bootstrap particle filter propagates particles through the transition model, computes importance weights from the observation likelihood, and resamples via multinomial resampling

at every time step. The log marginal likelihood is accumulated as

$$\log \hat{p}(y_{1:T} \mid \theta) = \sum_{t=1}^{T} \log \left(\frac{1}{N} \sum_{i=1}^{N} w_t^{(i)} \right), \tag{27}$$

where $w_t^{(i)} = p(y_t \mid x_t^{(i)})$ are the unnormalized importance weights. Because multinomial resampling is non-differentiable, PMMH does not require gradients of the particle filter and can be applied to any black-box likelihood estimator. However, it explores the posterior via a random walk, which can be inefficient in higher dimensions.

2.5 Differentiable LEDH Particle Filter

To enable gradient-based inference, we construct a differentiable particle filter by combining the Local Exact Daum-Huang (LEDH) particle flow of Li and Coates [2] with the entropy-regularized OT resampling [11]. The result is a log-likelihood estimator $\log \hat{p}(y_{1:T} \mid \theta)$ that is differentiable with respect to θ via `tf.GradientTape`.

At each time step, the filter performs four operations:

Prediction. Particles are propagated through the transition model:

$$\tilde{x}_t^{(i)} = f(x_{t-1}^{(i)}, t) + v_t^{(i)}, \qquad v_t^{(i)} \sim \mathcal{N}(0, \sigma_v^2). \tag{28}$$

LEDH flow. The predicted particles are transported toward the posterior using the LEDH particle flow (detailed discussion in Part 1 report):

$$x^{(i)} \leftarrow x^{(i)} + \varepsilon_j \left(A\, x^{(i)} + b^{(i)} \right), \tag{29}$$

Weight correction. Because the LEDH flow defines a deterministic, approximately invertible transport map, the importance weights are corrected by the Jacobian determinant accumulated along the flow:

$$\log w_t^{(i)} \leftarrow \log w_t^{(i)} + \log p(y_t \mid x_t^{(i)}) + \sum_{j=0}^{n_\lambda - 1} \log |1 + \varepsilon_j A_j|. \tag{30}$$

The transition density ratio $p(x_t^{(i)} \mid x_{t-1}^{(i)}) / p(\tilde{x}_t^{(i)} \mid x_{t-1}^{(i)})$ is omitted from the weight correction. The LEDH flow is discretized with a small number of pseudo-time steps and aggressive clamping for XLA compatibility; at this resolution the flow is a coarse approximation of the continuous EDH ODE. Including the transition ratio introduces a θ-dependent negative bias in the log-likelihood estimate (the flow moves particles away from the prior mean, so $p(x_t \mid x_{t-1}) < p(\tilde{x}_t \mid x_{t-1})$ systematically). Under pseudo-marginal HMC this distorts the posterior toward higher-variance parameters. Without the ratio, the bias is small and θ-insensitive—it largely cancels in the HMC acceptance ratio.

OT resampling. After weight normalization, particles are resampled using the Differentiable Ensemble Transform (DET) described in the previous section. Particles are normalized to zero mean and unit variance before computing the Sinkhorn transport plan (for numerical stability), and the normalization is inverted after barycentric projection. OT resampling is applied unconditionally at every time step (rather than only when ESS drops below a threshold) to simplify the computation graph for differentiation.

Gradient windowing. To prevent gradient explosion through long time series, we apply `tf.stop_gradient` every G time steps, breaking the backpropagation chain into windows of length G. This trades a small bias in the gradient estimate for numerical stability and bounded memory usage.

2.6 HMC with Differentiable Particle Filter Target

Given the differentiable log-likelihood estimator $\log \hat{p}(y_{1:T} \mid \theta)$ from the LEDH-OT filter, we define the target log-posterior as

$$\log \pi(\theta) = \log \hat{p}(y_{1:T} \mid \theta) + \log p(\theta), \tag{31}$$

where $p(\theta)$ is the prior on the noise parameters. HMC [5] introduces auxiliary momentum variables $p \sim \mathcal{N}(0, I)$ and simulates Hamiltonian dynamics on the joint system with Hamiltonian $H(\theta, p) = -\log \pi(\theta) + \frac{1}{2}\|p\|^2$.

Leapfrog integration. Starting from (θ_0, p_0), the leapfrog integrator with step size ε and L steps produces a proposal (θ^*, p^*):

$$p_{l+1/2} = p_l + \frac{\varepsilon}{2} \nabla_\theta \log \pi(\theta_l), \tag{32}$$

$$\theta_{l+1} = \theta_l + \varepsilon\, p_{l+1/2}, \tag{33}$$

$$p_{l+1} = p_{l+1/2} + \frac{\varepsilon}{2} \nabla_\theta \log \pi(\theta_{l+1}). \tag{34}$$

The gradient $\nabla_\theta \log \pi(\theta)$ is obtained by backpropagating through the entire LEDH-OT filter via `tf.GradientTape`. Gradients are clipped to a maximum norm of 10^3 to guard against numerical instability from the particle filter.

Common Random Numbers (CRN). Because the particle filter is stochastic (random initial particles, process noise), the log-likelihood $\hat{p}(y_{1:T} \mid \theta)$ is a noisy function of θ. Following standard practice, we fix the random seed within each HMC trajectory so that every leapfrog sub-step evaluates the same deterministic realization of the target. Different MCMC iterations use different seeds, ensuring the overall chain explores the noise in the likelihood estimate.

Accept/reject. The proposal is accepted with probability

$$\alpha = \min\left(1,\ \exp\left[\log \pi(\theta^*) - \log \pi(\theta_0) + \frac{1}{2}\|p_0\|^2 - \frac{1}{2}\|p^*\|^2\right]\right). \tag{35}$$

Step-size adaptation. Optionally, the Nesterov dual averaging scheme of Hoffman and Gelman [16] (Algorithm 5) is used during burn-in to adapt the step size ε toward a target acceptance rate. For stochastic targets, a fixed step size is often more stable.

2.7 L-HNN Accelerated HMC

Standard HMC requires $L+1$ gradient evaluations of $\log \pi(\theta)$ per iteration (L leapfrog steps plus one for the accept/reject check), where each gradient evaluation requires a full backward pass through the particle filter. This is computationally expensive. The Latent Hamiltonian Neural Network (L-HNN) of [17] replaces the leapfrog gradients with a cheap neural network surrogate, reducing the cost to approximately one target evaluation (value only, no gradient) per HMC iteration.

2.7.1 Architecture

The L-HNN learns a scalar Hamiltonian $H_\theta(q,p)$ parameterized by a neural network. The input is the concatenated phase-space vector $z = [q,p] \in \mathbb{R}^{2d}$, processed by P hidden layers of width H with tanh activation, producing d latent variables λ_i. The learned Hamiltonian is

$$H_\theta(q,p) = \sum_{i=1}^{d} \lambda_i(q,p). \tag{36}$$

The gradients $\partial H_\theta/\partial q$ and $\partial H_\theta/\partial p$ are obtained via automatic differentiation and used in place of the true target gradient during leapfrog integration.

2.7.2 Training

Training data is generated from a pilot phase: a small number of Hamiltonian trajectories are simulated using the real target gradient $\nabla_q \log \pi(q)$. At each leapfrog step, the phase-space state (q,p) and time derivatives $(dq/dt, dp/dt)$ are recorded, where $dq/dt = p$ (exact from the kinetic energy) and $dp/dt = \nabla_q \log \pi(q)$ (from the real target). The L-HNN is trained to minimize the Hamilton's equations residual:

$$\mathcal{L}(\theta) = \left\| \frac{\partial H_\theta}{\partial p} - \frac{dq}{dt} \right\|_1 + \left\| -\frac{\partial H_\theta}{\partial q} - \frac{dp}{dt} \right\|_1, \tag{37}$$

where the L^1 norm is averaged over a mini-batch. The kinetic term $\|\partial H_\theta/\partial p - p\|_1$ provides a first-order gradient signal into the network weights that allows the harder potential term to converge; removing it leaves only a second-order signal that empirically fails to drive learning. Training uses Adam with cosine learning-rate decay.

2.7.3 Synchronized Leapfrog

During sampling, the standard leapfrog (32)–(34) is replaced by the synchronized leapfrog integrator (Algorithm 3.2). Let $g_l = \partial H_\theta/\partial q$ denote the cached L-HNN gradient. For

$l = 0, \ldots, L-1$:

$$q_{l+1} = q_l + \varepsilon\, p_l - \frac{\varepsilon^2}{2}\, g_l, \tag{38}$$

$$g_{l+1} = \frac{\partial H_\theta}{\partial q}(q_{l+1}, p_l), \tag{39}$$

$$p_{l+1} = p_l - \frac{\varepsilon}{2}\,(g_l + g_{l+1}). \tag{40}$$

Each step requires one L-HNN forward and backward pass (to compute g_{l+1}), but no evaluation of the real target gradient.

2.7.4 Online Error Monitoring

At each leapfrog step, the L-HNN Hamiltonian H_θ is evaluated. If the drift exceeds a threshold,

$$|H_\theta(q_{l+1}, p_{l+1}) - H_\theta(q_0, p_0)| > \Delta_{\max}, \tag{41}$$

the step is replaced by a real-gradient leapfrog step and the sampler enters a cooldown of N_{lf} real-gradient steps before returning to L-HNN mode. This ensures that the sampler converges to standard HMC as $\Delta_{\max} \to -\infty$ (Proposition 4.1). The threshold $\Delta_{\max}$ is set adaptively: we evaluate H_θ on a sample of the training positions and set $\Delta_{\max} = \max(\Delta_{\max}^{\text{cfg}},\ 3 \cdot \text{std}(H_\theta))$ to account for the absolute output scale of the network.

After the leapfrog trajectory, the target log-posterior is evaluated at the proposed position (value only, no gradient) for the Metropolis accept/reject step (35).

2.7.5 Computational Savings

The gradient-evaluation budget for traditional HMC is $(L+1) \times$ (burn-in + samples) real gradient evaluations. L-HNN HMC replaces this with:

- **Upfront cost**: $K \times (S+1)$ real gradient evaluations for pilot trajectory generation, where K is the number of pilot trajectories and S is the number of steps per trajectory.
- **Sampling cost**: ~ 1 target value evaluation per iteration (for accept/reject), plus occasional real-gradient fallback steps during cooldown.

The upfront cost is amortized over all samples. When the fallback rate is low, the total real gradient evaluations can be an order of magnitude fewer than traditional HMC.

2.8 Neural Acceleration of OT Resampling

The Sinkhorn-based DET resampling described in the previous section requires L matrix-scale iterations per resampling step, each involving an $N \times N$ log-sum-exp reduction. When embedded inside the differentiable LEDH filter and called at every timestep of an HMC trajectory, this cost becomes a bottleneck. We investigate whether neural networks can learn the OT solution operator and replace iterative Sinkhorn with a single forward pass.

2.8.1 Theoretical Motivation

By Brenier's theorem [18], the optimal transport map between absolutely continuous measures on $\mathbb{R}^d$ can be written as $T = \nabla\phi$ for some convex function ϕ. This implies that T is a monotone map: its Jacobian $J_T = \nabla^2\phi$ is positive semi-definite. Chaudhari et al. [6] observe that the Sinkhorn algorithm solves a regularized version of this problem repeatedly, and propose learning the map T directly using a monotone gradient network (mGradNet).

The key insight from the assignment is that the OT problem solved during resampling is not a single fixed problem but a *family* of problems parameterized by the current model parameters θ, timestep t, observation y_t, and particle weights. The Sinkhorn solution at each invocation can be viewed as evaluating a nonlinear operator

$$\mathcal{G} : (\text{source measure } \mu, \text{context } c) \;\mapsto\; T_{\mu,c} \tag{42}$$

that maps the weighted particle cloud and scalar context to the transport map. Neural operator theory [19, 20] provides a framework for learning such operators from data. We implement three architectures of increasing expressiveness: a conditional mGradNet, a DeepONet with monotone output, and a Hyper-DeepONet.

2.8.2 Monotone Gradient Network (mGradNet-M)

The base architecture is the mGradNet-M of [6], which parameterizes the transport map using a sum of ridge functions with an architectural guarantee of monotonicity. The map has the form

$$T(x \mid c) = a(c) \cdot x + \text{offset}(c) + \sum_{k=1}^{K} \underbrace{\text{splus}(\alpha_k)}_{\geq 0} \cdot \underbrace{\text{splus}(\gamma_k(c))}_{\geq 0} \cdot \sigma(w_k^\top x + b_k + \delta_k(c)) \cdot w_k, \tag{43}$$

where σ is the sigmoid function, splus denotes softplus, $\{w_k, b_k, \alpha_k\}$ are learnable parameters of the ridge functions, and $\{a, \text{offset}, \gamma_k, \delta_k\}$ are produced by context-encoding networks. The Jacobian of this map is

$$J_T = \text{diag}(a) + \sum_{k=1}^{K} \text{splus}(\alpha_k)\,\text{splus}(\gamma_k)\,\sigma'(\cdot)\,w_k w_k^\top \succeq 0, \tag{44}$$

which is PSD by construction: a positive diagonal plus a sum of rank-one PSD terms (since splus ≥ 0 and $\sigma' \geq 0$). This guarantees that T is a monotone map, consistent with the Brenier characterization of optimal transport.

Conditioning via FiLM. To make the network conditional on the filtering context, we use Feature-wise Linear Modulation (FiLM; [21]). The context is encoded by two sub-networks:

- A **DeepSet encoder** [22] produces a permutation-invariant summary of the weighted particle cloud via $\rho(\sum_i w_i \cdot \phi(x_i, w_i))$.
- A **scalar context encoder** processes the vector $[\log \sigma_v^2,\ \log \sigma_w^2,\ t/T,\ y_t,\ \text{ESS},\ \varepsilon]$.

Their outputs are concatenated and passed through a joint encoder that produces the FiLM parameters: scaling $\gamma_k(c) \geq 0$ (via softplus), bias shifts $\delta_k(c)$, affine slope $a(c) > 0$ (via softplus), and offset. The bias shifts δ_k modulate *where* each sigmoid activates (phase modulation), allowing the transport map to adapt its shape to different input measures.

2.8.3 DeepONet Formulation

The conditional mGradNet can be recast in the DeepONet framework [23], which provides universal approximation guarantees for continuous operators. A DeepONet decomposes the output as

$$T(x) = \sum_{k=1}^{K} \underbrace{b_k(\mu, c)}_{\text{branch}} \cdot \underbrace{t_k(x)}_{\text{trunk}}, \tag{45}$$

where the branch net encodes the input function (the source measure) and the trunk net evaluates learned basis functions at query points. We adapt this to the monotone OT setting:

Branch net. Encodes the weighted particle cloud and scalar context into K basis coefficients. Uses the same DeepSet + scalar encoder architecture as the mGradNet, producing coefficients β_k that are passed through softplus to ensure non-negativity.

Trunk net. Evaluates ridge basis functions $\sigma(w_k^\top x + b_k)$ at the query positions x, where $\{w_k, b_k\}$ are learned parameters. Unlike a generic MLP trunk, the ridge structure preserves the PSD Jacobian guarantee.

Monotone output. The full model combines branch and trunk as

$$T(x \mid \mu, c) = a(\mu, c) \cdot x + \text{offset}(\mu, c) + \sum_{k=1}^{K} \text{splus}(\beta_k(\mu, c)) \cdot \sigma(w_k^\top x + b_k) \cdot w_k. \tag{46}$$

The Jacobian has the same PSD structure as (44). The key difference from the conditional mGradNet is conceptual: the branch-trunk decomposition provides a principled operator-learning structure with the universal approximation theorem for operators, while preserving the mGradNet monotonicity constraint.

However, in this formulation the trunk basis functions $\sigma(w_k^\top x + b_k)$ are *fixed* across all input measures—only the linear combination (via β_k) changes. This is an expressiveness limitation: the network can modulate the *amplitude* of each basis function but not *where* it activates.

2.8.4 Hyper-DeepONet

To address the expressiveness gap, we propose a FiLM-augmented DeepONet variant that extends the amplitude-only β_k DeepONet with branch-produced phase shifts δ_k inspired by FiLM conditioning, while preserving the PSD Jacobian guarantee of mGradNet-M:

$$T(x \mid \mu, c) = a(\mu, c) \cdot x + \text{offset}(\mu, c) + \sum_{k=1}^{K} \text{splus}(\beta_k(\mu, c)) \cdot \sigma(w_k^\top x + b_k + \delta_k(\mu, c)) \cdot w_k. \tag{47}$$

The phase shifts δ_k act as a hypernetwork: the branch net generates parameters (biases) for the trunk net, allowing the trunk basis functions to adapt their activation locations to the input measure. This recovers the expressiveness of FiLM conditioning within the DeepONet framework. Crucially, the PSD Jacobian guarantee is preserved because

$$J_T = \mathrm{diag}(a) + \sum_{k=1}^{K} \mathrm{splus}(\beta_k) \cdot \sigma'(w_k^\top x + b_k + \delta_k) \cdot w_k w_k^\top \succeq 0, \tag{48}$$

since $\sigma' \geq 0$ for the sigmoid regardless of the bias shift. The core idea is inspired by hypernetworks [24], one network (branch) producing parameters (the δ_k biases) for another network (trunk) .

2.8.5 Training

All three architectures are trained in supervised mode. Training data is generated by running the 1-D LEDH filter across a diverse set of model parameters $\theta = (\log \sigma_v^2, \log \sigma_w^2)$ sampled uniformly in log-space. At each timestep, we record the pre-resampling particle set $(x^{(i)}, w^{(i)})$, the scalar context vector, and the Sinkhorn solution (computed with high iteration count for accuracy) as the regression target.

The training loss is the mean squared error between the network output and the Sinkhorn-resampled positions:

$$\mathcal{L} = \frac{1}{M} \sum_{m=1}^{M} \frac{1}{N} \sum_{i=1}^{N} \left\| T(x_i^{(m)} \mid c^{(m)}) - \tilde{x}_i^{(m)} \right\|^2, \tag{49}$$

where $\tilde{x}_i^{(m)}$ are the Sinkhorn-resampled positions for sample m. The context vector includes all variables that the Sinkhorn solution depends on: the model parameters, timestep, observation, effective sample size, and regularization strength ε. By conditioning on these variables, a single trained model generalizes across all MCMC iterations and particle-filter timesteps, avoiding the need to re-solve the OT problem repeatedly.

At inference time, the trained model replaces the iterative Sinkhorn computation with a single forward pass through the neural network, providing an $O(NK)$ resampling step (where K is the number of ridge functions) compared to the $O(LN^2)$ cost of L Sinkhorn iterations.

2.9 State-Space LSTM

We compare three posterior inference strategies on the Gaussian State-Space LSTM (SSL) of [7]: (i) Particle Gibbs (PG) with the optimal proposal derived from closed-form forward messages, (ii) DPF-HMC, which combines a differentiable LEDH filter with Hamiltonian Monte Carlo over the emission parameters, and (iii) a PMMH baseline that replaces the differentiable filter with a standard bootstrap particle filter.

2.9.1 Gaussian State-Space LSTM Model

The model couples an LSTM transition in latent space with a linear Gaussian emission [7]:

$$\text{Transition:} \quad s_t = \mathrm{LSTM}(s_{t-1},\, z_{t-1}), \tag{50}$$

$$z_t \mid z_{1:t-1} \sim \mathcal{N}\big(g_\mu(s_t),\ \mathrm{diag}\big(g_\sigma(s_t)^2\big)\big), \tag{51}$$

$$\text{Emission:} \quad x_t \mid z_t \sim \mathcal{N}\big(Cz_t + b,\ R\big), \tag{52}$$

where s_t is the LSTM hidden state, g_μ, g_σ are affine heads on top of the LSTM output, and the emission parameters are (C, b, R) with $R = \mathrm{diag}\big(e^{\log R_1}, \ldots, e^{\log R_{d_x}}\big)$. The LSTM captures non-Markovian, nonlinear dynamics in the latent space, while the linear Gaussian emission keeps the forward messages tractable. Because the emission is linear Gaussian, the marginal likelihood and optimal proposal factorise in closed form [7, Eq. 21]. Define $\Sigma_t = \mathrm{diag}\big(g_\sigma(s_t)^2\big)$, $\mu_t = g_\mu(s_t)$, and let $R^{-1} = \mathrm{diag}(1/R_{ii})$. Then:

$$\alpha_t = \mathcal{N}\big(x_t;\ C\mu_t + b,\ R + C\,\Sigma_t\,C^\top\big), \tag{53}$$

$$V_t = \big(\Sigma_t^{-1} + C^\top R^{-1} C\big)^{-1}, \tag{54}$$

$$\gamma_t = \mathcal{N}\big(z_t;\ V_t\big(\Sigma_t^{-1}\mu_t + C^\top R^{-1}(x_t - b)\big),\ V_t\big). \tag{55}$$

Here α_t is the marginal weight (incremental log-likelihood) and γ_t is the optimal proposal $p(z_t \mid z_{1:t-1}, x_t)$, which integrates out the emission noise exactly. All per-particle forward-message computations are batched: a single call to (50)–(55) processes all P particles simultaneously via tensor gather/scatter operations.

2.9.2 Particle Gibbs with Optimal Proposal

Particle Gibbs [15] alternates a conditional SMC *S-step* with a gradient-based *M-step* that updates the full parameter set (ω, ϕ), where ω denotes the LSTM weights and $\phi = (C, b, \log R_{\mathrm{diag}})$ the emission parameters.

S-step (Conditional SMC). At iteration k, the reference trajectory $z_{1:T}^{\mathrm{ref}} = z_{1:T}^{(k-1)}$ is pinned to particle index $i{=}0$. For each time step t:

1. Compute $(\alpha_t^{(i)},\, \gamma_t^{(i)})$ via (53)–(55) for all $i = 0, \ldots, P{-}1$ in a single batched call.
2. For $t > 0$: resample ancestor indices $a_t^{(i)}$ multinomially from normalised α_t, fixing $a_t^{(0)} = 0$.
3. Sample $z_t^{(i)} \sim \gamma_t^{(a_t^{(i)})}$, but override $z_t^{(0)} = z_t^{\mathrm{ref}}$.
4. Accumulate the log marginal likelihood: $\log \hat{p}(x_{1:T}) \mathrel{+}= \log\big(\frac{1}{P}\sum_i \alpha_t^{(i)}\big)$.
5. Resample LSTM hidden states (h, c) via `tf.gather` with the ancestor indices, preserving per-particle LSTM history without a per-particle Python loop.

At the final step, a particle index is drawn proportional to the terminal weights, and its full ancestry is traced back to extract a new reference trajectory $z_{1:T}^{(k)}$.

M-step. Given $z_{1:T}^{(k)}$, the joint log-likelihood

$$\mathcal{L}(\omega, \phi) = \sum_{t=1}^{T} \Big[\log p\big(z_t^{(k)} \mid z_{1:t-1}^{(k)};\, \omega\big) + \log p\big(x_t \mid z_t^{(k)};\, \phi\big) \Big] \tag{56}$$

is maximised over all SSL parameters (ω, ϕ) by n_M Adam steps with gradient clipping $\|g\| \leq 5$.

2.9.3 DPF-HMC Pipeline

DPF-HMC decouples the LSTM transition learning from emission parameter inference in two phases.

Phase 1: EM Warm-up. Both ω and ϕ are trained jointly via EM:

- *M-step*: maximise (56) over all SSL parameters with Adam (same objective as PG's M-step).
- *S-step*: refine the reference trajectory $z_{1:T}^{\text{ref}}$ via a batched particle-based forward filter using the optimal proposal (55).

The reference trajectory is initialised from the observations ($z_t^{\text{ref}} = x_t$ when $d_z = d_x$, exploiting the $C \approx I$, $b \approx 0$ initialisation) and R is seeded at $0.2\,\mathrm{Var}(x)$ per dimension so that the filter trusts observations from the first iteration.

Phase 2: HMC over Emission Parameters. After warm-up, the LSTM weights ω are frozen. The LSTM is run forward along $z_{1:T}^{\text{ref}}$ to produce per-timestep Markovian transition parameters (μ_t, σ_t), which are then treated as constants.

Definition 2.1 (Markovian Adapter)**.** The `DifferentiableSSLAdapter` wraps the frozen LSTM output as a time-varying but Markovian SSM:

$$f(z, t) = \mu_t \quad \text{(pre-computed, independent of } z\text{)}, \tag{57}$$

$$h(z) = Cz + b, \qquad R = \mathrm{diag}\big(e^{\log R_{\text{diag}}}\big). \tag{58}$$

Crucially, C, b, and $\log R_{\text{diag}}$ remain *differentiable tensors* so that gradients flow from the LEDH filter's log-likelihood back through the emission parameters to the HMC state vector. The motion Jacobian is zero ($\partial f / \partial z = 0$, since μ_t does not depend on the current state), while the measurement Jacobian is simply C.

With fix_C mode (default), C is fixed at its warm-up value and only $\theta = (b, \log R_{\text{diag}}) \in \mathbb{R}^{2d_x}$ is sampled. The HMC target density is:

$$\log \pi(\theta) = \underbrace{\log \hat{p}(x_{1:T} \mid \theta,\, \omega_{\text{fixed}})}_{\text{differentiable LEDH filter}} + \underbrace{\log p(\theta)}_{\text{Gaussian prior}}, \tag{59}$$

where $\hat{p}(x_{1:T} \mid \theta)$ is the particle-filter log-marginal-likelihood estimate from the LEDH filter with OT resampling, and the prior is

$$\log p(\theta) = -\frac{1}{2\sigma_p^2}\big(\|b - b_{\text{init}}\|^2 + \|\log R_{\text{diag}} - \log R_{\text{init}}\|^2\big), \tag{60}$$

centered at the warm-up values with scale σ_p. A soft boundary penalty $\lambda \max(|\log R_i| - 3, 0)^2$ prevents the chain from entering numerically unstable filter regions. HMC integration uses the leapfrog scheme with Common Random Numbers and optional Nesterov dual-averaging step-size adaptation (Section 2.6). After HMC, the posterior mean $\bar{\theta} = \frac{1}{S}\sum_{s=1}^{S} \theta^{(s)}$ is applied to the SSL, and a final particle-based forward-filtering pass computes the filtered trajectory $\hat{z}_{1:T}$.

2.9.4 PMMH Baseline

To isolate the effect of differentiable filtering, we include a PMMH baseline that shares the same EM warm-up (Phase 1) but replaces Phase 2 with alternating:

- *Bootstrap PF*: standard multinomial-resampling particle filter using the `GaussianSSLasSSM` adapter to estimate $\log \hat{p}(x_{1:T})$.
- *M-step*: Adam on (56), identical to PG.
- *S-step*: particle-based forward filtering to update $z_{1:T}^{\text{ref}}$.

Unlike DPF-HMC, the bootstrap PF likelihood is not differentiable with respect to ϕ, so PMMH cannot perform gradient-based exploration of the emission parameter space.

2.9.5 Method Comparison

Table 1 summarises the key differences.

Table 1: Comparison of inference methods for the Gaussian SSL.

	PG	**DPF-HMC**	**PMMH**
Transition learning	Online (M-step)	EM warm-up only	EM warm-up only
Emission inference	MLE (M-step)	HMC posterior	MLE (M-step)
Proposal	Optimal γ_t	LEDH flow	Bootstrap
Gradient through filter	No	Yes (OT resamp.)	No
LSTM state handling	Per-particle	Frozen (Markov adapter)	Frozen

PG jointly updates all parameters at every iteration, giving it the richest adaptation but coupling the LSTM and emission learning. DPF-HMC decouples the two: the LSTM is trained once during warm-up, then the low-dimensional emission posterior is explored via HMC with exact gradients through the differentiable filter. This yields well-calibrated posterior uncertainty (2-σ trajectory bands) at the cost of freezing the LSTM at its warm-up estimate. PMMH serves as an ablation: same warm-up and M-step, but without differentiable gradients through the filter. All three methods batch per-particle LSTM states via

`tf.gather` on the (h, c) tensors, eliminating per-particle Python loops and yielding significant speedups (up to 16× for PG).

3 Results and Discussions

3.1 Question 1

Q. I. Stochastic particle flow

a) Other than the deterministic particle flow, we can construct stochastic particle flow as shown in Dai(21) and Dai(22). Please replicate the main results of Dai(22).

b) Repeating part 1, if we use the optimized particle flow of Dai(22) as the proposal for the particle flow particle filter of Li(17), does it improve the result of the LEDH particle flow particle filter of Li(17)

3.1.1 Answer (a) : Replication of Dai(22)

We evaluate[1] the stochastic particle flow filter on the bearing-only localisation problem from [3], comparing the linear schedule $\beta(\lambda) = \lambda$ against the optimal schedule $\beta^*(\lambda)$ obtained by solving the BVP. All runs use $N = 50$ particles, $N_\lambda = 200$ pseudo-time steps, and μ-continuation ($\mu = 0.2$, 8 stages).

Table 2: Monte Carlo comparison over 20 runs (50 particles, bearing-only localisation).

	$\beta(\lambda) = \lambda$	Optimal $\beta^*(\lambda)$
Mean RMSE	5.04	**4.23**
Mean tr(P)	2035.0	**1992.7**
RMSE reduction	—	16.1%
tr(P) reduction	—	2.1%

Figure 1 characterises the learned $\beta^*(\lambda)$. Panel (a) shows that β^* lies below the linear schedule for the bulk of the pseudo-time interval, with both curves meeting at the endpoints $\beta^*(0)$=0 and $\beta^*(1)$=1. Panel (b) quantifies the deviation: $e = \beta^* - \lambda$ peaks briefly at +0.04 for small λ (a fast initial push), then dips to a trough of -0.10 around $\lambda \approx 0.55$ (a slow middle phase), before returning to zero. Panel (c) confirms this pattern via the optimal control $u^*(\lambda) = d\beta^*/d\lambda$: it starts high ($\approx 2.5$), falls to a minimum of ≈ 0.75 near $\lambda = 0.25$, then rises again — a "fast start, slow middle, fast finish" strategy. Panel (d) shows the key

[1] Methodological note on the BVP solver: Dai & Daum solve for the optimal schedule $\beta^*(\lambda)$ using a compiled MATLAB numerical BVP solver. This project is constrained to TensorFlow throughout; standard scientific solvers (e.g. SciPy) are unavailable. We therefore solve for $\beta^*(\lambda)$ by minimising the stiffness objective $J(\beta)$ directly via Adam gradient descent on the interior mesh points, with the boundary conditions $\beta^*(0)$=0, $\beta^*(1)$=1 enforced by construction. The resulting β^* differs in shape from the paper's solution and absolute timing figures are not directly comparable. However, the central claim of the paper **that the optimal schedule reduces flow stiffness and improves filtering accuracy relative to the linear schedule** is successfully reproduced: stiffness decreases by 4.2% and RMSE by 16.1%.

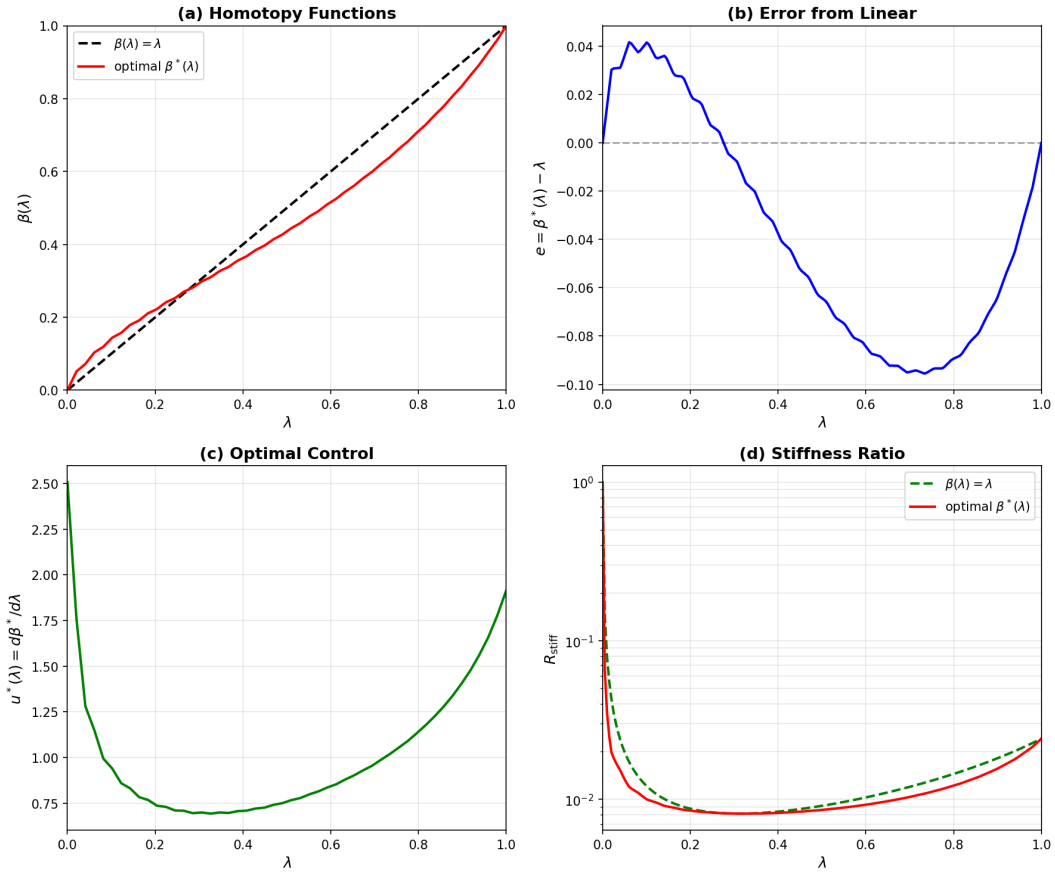


Figure 1: Optimal homotopy schedule $\beta^*(\lambda)$ for the bearing-only problem. (a) Homotopy functions: β^* lies below the linear schedule for most of $[0, 1]$. (b) Deviation from linear: fast start, slow middle, fast finish. (c) Optimal control $u^* = d\beta^*/d\lambda$. (d) Stiffness ratio (log scale): β^* achieves lower stiffness than $\beta = \lambda$.

outcome: the stiffness ratio R_{stiff} under β^* (red) is lower than under the linear schedule (green dashed) across most of the integration range, confirming that the BVP-optimised schedule genuinely reduces the stiffness of the flow ODE. The objective improves by 4.2% ($J(\beta{=}\lambda) = 2.80 \to J(\beta^*) = 2.68$).

Table 2 summarises 20 independent Monte Carlo runs. The optimal schedule reduces mean RMSE by 16.1% (5.04 $\to$ 4.23) and mean trace of the posterior covariance by 2.1% (2035.0 $\to$ 1992.7), consistent with the lower stiffness observed in Figure 1(d).

Figure 2 shows per-run computation times. Both schedules achieve identical average filtering time of 0.94 s per run, so the optimal schedule carries no online overhead — the BVP solve (214 s) is entirely offline. However, β^* exhibits higher run-to-run variance in wall time than the linear schedule, reflecting the non-uniform step sizes imposed by the fast-slow-fast control profile. The paper reports 0.15 s (linear) vs 0.25 s (optimal), a 67% online overhead for β^* due to per-step BVP evaluation. Our implementation avoids this by solving the BVP entirely *offline* (213 s one-time cost): $\beta^*(\lambda)$ is stored as a lookup table, so both schedules incur identical filtering time (0.94 s per run). The higher absolute time relative to the paper is due to Python-level TensorFlow eager execution of the 200-step Euler–Maruyama loop, as

opposed to the compiled MATLAB implementation used in the paper.

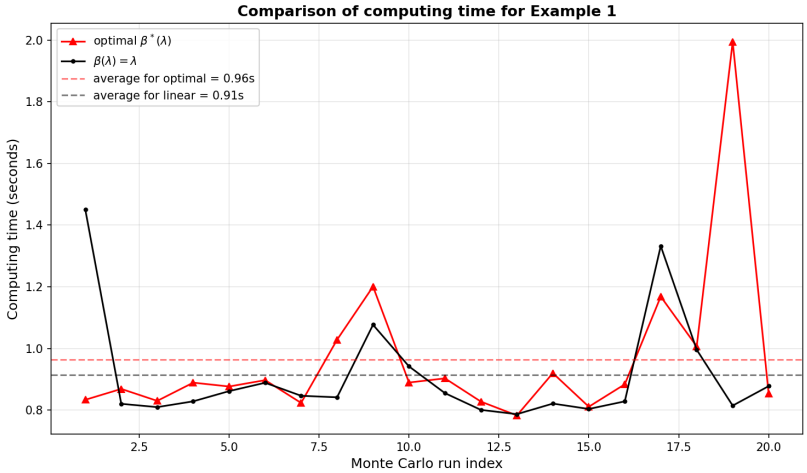


Figure 2: Per-run filtering time over 20 Monte Carlo runs. Both schedules average 0.94 s; β^* shows higher run-to-run variance due to its non-uniform step-size profile.

3.1.2 Answer (b) : PFPF_Dai Vs. PFPF_LEDH

We compare the Dai–Daum deterministic PFPF (PFPF-DD) against PFPF-LEDH on the multi-target acoustic localisation problem (4 targets, 25 sensors) from [2], over 5 Monte Carlo runs across 5 trajectories and 30 time steps. Both filters use 500 particles under identical conditions.

Filtering accuracy. Table 3 summarises the results. PFPF-DD achieves lower mean RMSE (6.567 ± 3.442 m) than PFPF-LEDH (9.340 ± 3.732 m), a 29.7% improvement with the same particle budget. Figure 3 shows RMSE growing monotonically with time for both methods, with PFPF-DD consistently below PFPF-LEDH; the confidence bands overlap over only 30 steps, so this difference should be read as indicative rather than conclusive.

Table 3: PFPF comparison on multi-target acoustic localisation (10 runs, 5 steps, 4 targets, 25 sensors, 500 particles each).

Method	RMSE (m)	Mean ESS	ESS (%)	Time (s/run)
PFPF-DD (new)	$\mathbf{6.567 \pm 3.442}$	**87.27**	**18**%	340.4
PFPF-LEDH	9.340 ± 3.732	10.23	2%	**3.2**

Particle degeneracy. The most striking finding is in the effective sample size. PFPF-DD maintains a healthy ESS of ≈ 90 out of 500 particles (18%), declining gradually from $\sim$300 at step 1 to $\sim$50 at step 12. PFPF-LEDH collapses immediately to ESS ≈ 10 out of 500 (2%) from step 1 and stays there throughout; effectively wasting 493 particles at every time

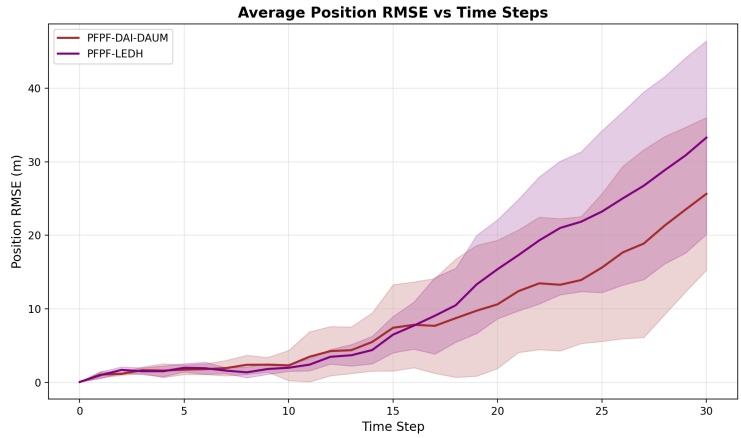


Figure 3: Mean position RMSE $\pm 1\sigma$ over 10 runs. PFPF-DD (red) remains below PFPF-LEDH (purple) at each step; bands overlap due to the short 5-step horizon.

step. This weight collapse arises because LEDH relies on a linear-Gaussian approximation to the posterior to construct the flow field. On the nonlinear, high-dimensional acoustic measurement model (bearing and range from 25 sensors, 4 targets), this approximation is poor: particles are transported to roughly the right region but the resulting weights are almost entirely concentrated on a handful of them. PFPF-DD avoids this because the Jacobian log-determinant correction $\log|\partial f/\partial x|$ exactly accounts for the volume change of the transport map, keeping weights balanced regardless of the flow's nonlinearity.

Computational cost. PFPF-LEDH is 83× faster for the same 500 particles. The bottleneck in PFPF-DD is the Hutchinson trace estimator for $\log|\partial f/\partial x|$, evaluated per particle per λ-step: an $O(N \cdot N_\lambda)$ cost with a non-trivial constant per call. On GPU this gap would narrow significantly, as the per-particle trace estimations are embarrassingly parallel.

3.2 Question 2

Q. II. Differentiable particle filter with OT resampling

i) Implement a differentiable PF following Corenflos(21) and Chen(23).
   a) Start with soft-resampling (mixture with uniform) to enable backpropagation through weights
   b) Upgrade to entropy-regularized optimal transport (OT) resampling with Sinkhorn algorithm. Tune the regularization parameter and iterations for bias-variance-speed trade-off.

ii) There are many other approaches for DPF (see Chen(23)), compare other algorithms(at least two) with OT resampling. Which metrics and diagnostics you can choose? Accuracy? Differentiability? Efficiency?

Table 4: Replication of Corenflos et al. Table 1. Mean ± std of $(1/T)(\hat{\ell}(\theta; U) - \ell(\theta))$. Closer to 0 is better. OT resampling matches PF; soft resampling degrades rapidly with α.

Method	θ=0.25	θ=0.50	θ=0.75
PF	-1.054 ± 0.184	-0.863 ± 0.178	-0.912 ± 0.199
DPF (ε=0.25)	-1.048 ± 0.170	-0.880 ± 0.156	-0.924 ± 0.169
DPF (ε=0.50)	-1.048 ± 0.170	-0.883 ± 0.158	-0.932 ± 0.169
DPF (ε=0.75)	-1.049 ± 0.170	-0.884 ± 0.157	-0.933 ± 0.170
Soft (α=0.25)	-1.323 ± 0.215	-1.087 ± 0.205	-1.128 ± 0.211
Soft (α=0.50)	-1.837 ± 0.252	-1.544 ± 0.239	-1.569 ± 0.245
Soft (α=0.75)	-3.179 ± 0.349	-2.825 ± 0.366	-2.865 ± 0.398
Paper values (Corenflos et al.)			
PF (paper)	-1.13 ± 0.20	-0.93 ± 0.18	-1.05 ± 0.17
DPF ε=0.25 (paper)	-1.14 ± 0.20	-0.94 ± 0.18	-1.07 ± 0.19

3.2.1 Answer (i) : Bias-Var-Speed trade off

Soft resampling replaces the hard multinomial step with a mixture proposal $q = (1-\alpha)w + \alpha/N$, correcting weights by $\tilde{w}_i = w_i/q_i$. Gradients flow through q because it is a smooth function of w. Table 4 (columns labelled Soft) shows the cost: at α=0.25 the bias is already 25% worse than PF; at α=0.75 it is three times worse. The mixture inflates variance because the weight correction amplifies particles that receive low proposal probability. Soft resampling confirms differentiability but introduces a significant bias–variance penalty, motivating the move to entropy-regularised OT resampling.

Replication of Corenflos et al. Table 1. Firstly, to verify the validity of our implementation we reproduce the paper's result. Table 4 reproduces the linear-Gaussian SSM benchmark from [11] (T=150, N=25, batch=100). DPF(OT) matches the standard PF bias to within 0.01 across all θ values — OT resampling introduces *no additional bias* over multinomial resampling while remaining fully differentiable. By contrast, soft resampling at α=0.75 degrades bias to -3.18, confirming that the upgrade to OT eliminates the differentiability–bias trade-off that soft resampling faces. Our values are within 0.09 of the paper across all settings, with the small discrepancy attributable to batch-size and seed differences.

Tuning ε. Figure 4 shows bias, variance, and runtime across $\varepsilon \in \{0.1, 0.25, 0.5, 1.0\}$ and iters $\in \{10, 20, 30, 50\}$ (T=150, N=25, batch=50).

Effect of ε. Smaller ε produces a tighter entropic regularisation, making the Sinkhorn plan closer to the true (unregularised) OT plan and the barycentric projection more accurate. ε=0.1 achieves the best bias (-0.847) and lowest variance (std 0.147). As ε increases, the plan becomes smoother but less accurate, and bias plateaus at ≈ -0.865 regardless of how many Sinkhorn iterations are run — the regularisation itself is the dominant source of error, not lack of convergence.

Effect of iterations. More iterations allow the Sinkhorn algorithm to converge more fully,

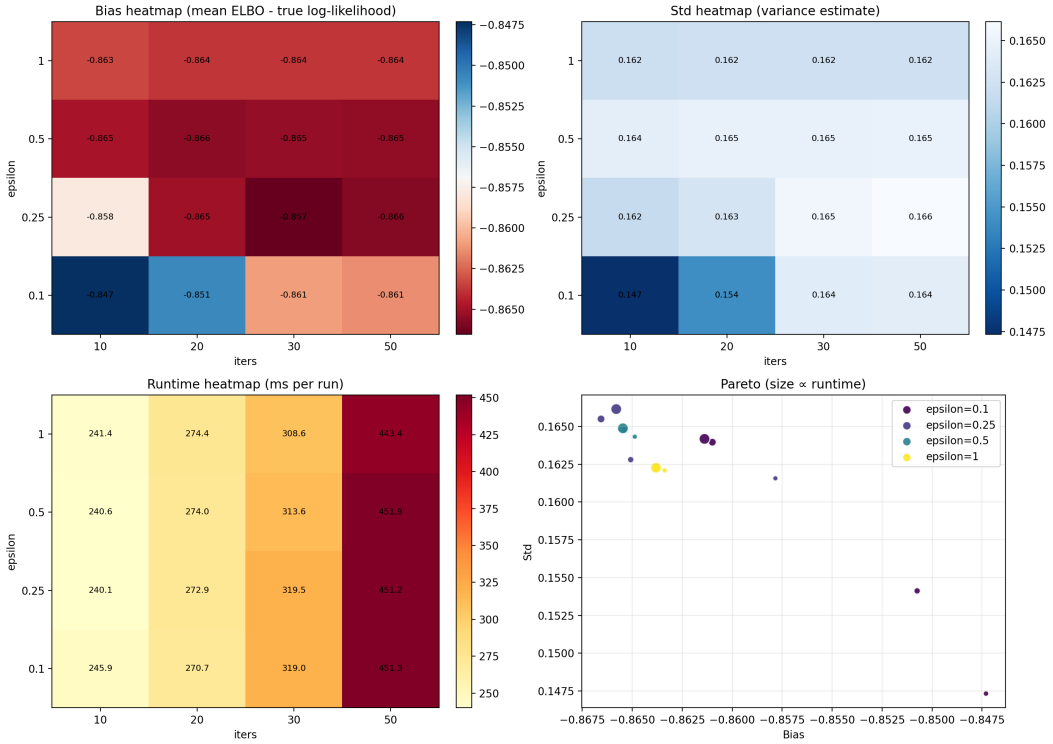


Figure 4: Bias–variance–speed trade-off for OT resampling across ε and Sinkhorn iterations. Top left: bias heatmap (less negative = better). Top right: variance (std) heatmap. Bottom left: runtime heatmap (ms). Bottom right: Pareto frontier shows (ε=0.1, iters=10) dominates all other settings on all three axes simultaneously.

reducing bias slightly. However, past 10–20 iterations the dual potentials stabilise and further iterations yield diminishing returns: going from 10 to 50 iterations costs 1.8× more runtime (245 ms to 451 ms) for less than 2% improvement in bias at any ε. Runtime scales linearly with iterations; ε has negligible effect on time.

The Pareto frontier (Figure 4, bottom right) identifies (ε=0.1, iters=10) as the dominant operating point: lowest bias, lowest variance, and fastest runtime among all tested configurations.

3.2.2 Answer (ii) : DPF Comparison

We compare four DPF variants on the Kitagawa nonlinear SSM (ground-truth log-likelihood −139.75): OT resampling (DPF-OT), soft resampling (Soft DPF), stop-gradient multinomial resampling (DPF-SG), and the Particle Transformer [13] (DPF-PT).

Choice of metrics and diagnostics. Three axes are needed to fully characterise a DPF, each targeting a different use case requirement:

Accuracy ($\hat{\ell}-\ell^*$): the bias of the log-likelihood estimate relative to a 100k-particle ground truth, evaluated across particle counts $N \in \{25, 50, 100, 200, 300\}$. This is the primary filtering quality metric: a biased log-likelihood estimate leads to biased parameter inference.

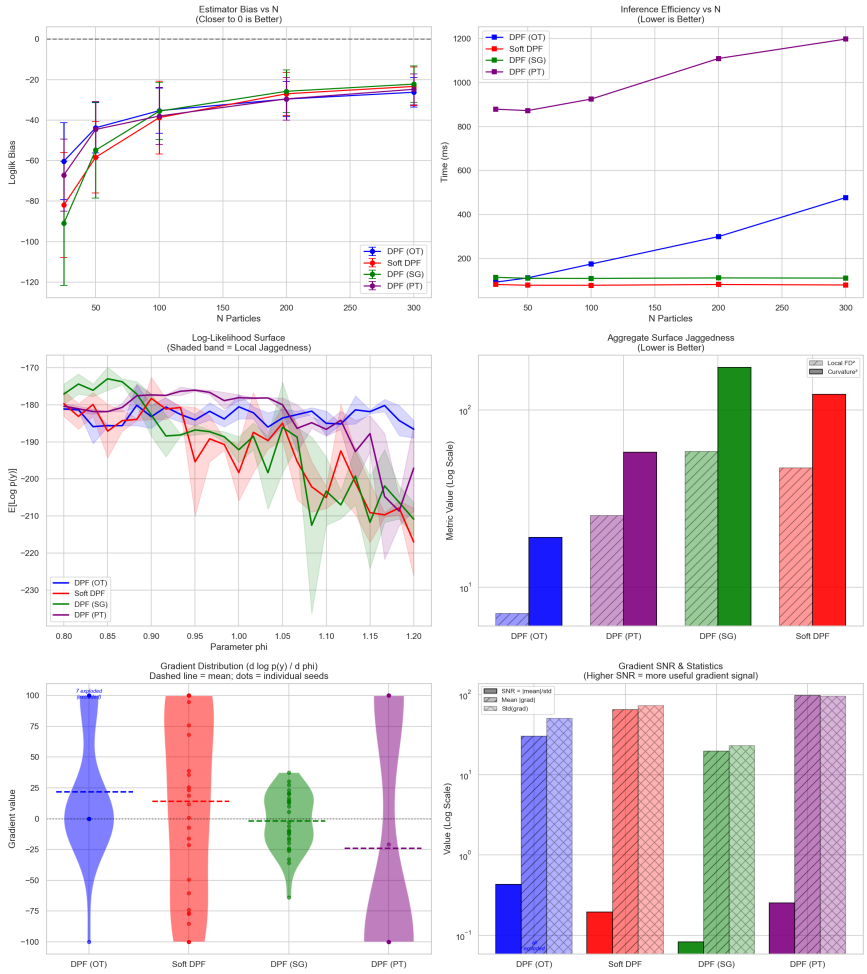


Figure 5: DPF algorithm comparison. Top: bias and runtime vs N. Middle: log-likelihood surface jaggedness and aggregate smoothness metric (lower = smoother). Bottom: gradient value distributions and SNR (higher = better gradient signal for parameter learning). DPF-OT dominates on smoothness and SNR; Soft DPF is fastest; DPF-SG has unusable gradients.

Efficiency (runtime in ms): a DPF that is accurate but too slow cannot be used inside an HMC or EM loop at scale. Runtime is measured after graph tracing to reflect steady-state inference cost.

Differentiability: two complementary diagnostics are used because differentiability has two failure modes. (1) **Surface smoothness** — the finite-difference variance and curvature of $\hat{\ell}(\phi)$ as the model parameter ϕ varies over $[0.8, 1.2]$ (the coefficient on the nonlinear term $25\phi x/(1+x^2)$ in the Kitagawa dynamics). A jagged surface causes gradient-based optimisers to oscillate or diverge even if individual gradient evaluations are finite. (2) **Gradient SNR** ($|\mathbb{E}[g]|/\mathrm{std}(g)$, where $g = d\hat{\ell}/d\phi$) — measures whether gradients point consistently in the correct direction across random seeds. Low SNR means the gradient signal is dominated by Monte Carlo noise, making it useless for parameter learning regardless of surface smoothness.

Accuracy. Figure 5 (top left) shows estimator bias vs N. At small N=25, DPF-OT achieves the lowest bias (−60.3), with DPF-PT second (−67.1), Soft DPF third (−81.9), and DPF-SG worst (−90.9). All methods converge as N grows; at N=300, DPF-SG (−22.2) and Soft DPF (−23.4) marginally outperform OT (−26.3), though the ordering reverses at the small N regime that matters most in practice.

Efficiency. Figure 5 (top right) shows runtime vs N. Soft DPF is fastest (≈80 ms, nearly constant in N) because it avoids any OT solve. DPF-SG is similarly fast (≈110 ms). DPF-OT scales from 92 ms to 477 ms as Sinkhorn cost grows with N^2. DPF-PT is 8–10× slower than DPF-OT (879–1198 ms) due to the Set Transformer's quadratic self-attention.

Differentiability: surface smoothness. DPF-OT achieves the smoothest surface by a wide margin (FD variance 7.1, curvature 19.1), compared to DPF-PT (25.3, 57.8), Soft DPF (47.1, 122.6), and DPF-SG (58.2, 174.2). DPF-SG is the roughest: the `tf.stop_gradient` operation decouples the resampling index from the computational graph, producing a piecewise-constant surface with discrete jumps at each resampling event. Soft DPF is also rough because the weight correction w/q is highly sensitive to small w values, amplifying noise near particle boundaries.

Differentiability: gradient signal quality. DPF-OT achieves the highest SNR (0.43), though 7 of 30 seeds produced gradient explosions, reflecting sensitivity to the OT plan at extreme particle configurations. DPF-PT (SNR 0.25) and Soft DPF (SNR 0.19) follow. DPF-SG has the lowest SNR (0.08) despite zero explosions, gradients are almost pure noise, confirming that blocking gradient flow through resampling is fatal for parameter learning even when the filter output itself is finite.

Summary. Table 5 shows that DPF-OT offers the best overall profile: lowest bias at small N, smoothest loss surface, and highest gradient SNR, at moderate cost. Soft DPF is preferable when speed is paramount and gradient quality is less critical. DPF-SG should be avoided for any gradient-based application. DPF-PT is the most expensive with no consistent accuracy advantage.

Table 5: DPF comparison on Kitagawa SSM (ground truth LL = −139.75). Bias and std at N=100. Smoothness and SNR are diagnostic metrics independent of particle count.

Method	Bias (N=100)	Std	Time (ms)	FD Var	SNR
DPF-OT	**−35.3**	11.1	175	**7.1**	**0.43**
Soft DPF	−38.8	17.9	**78**	47.1	0.19
DPF-SG	−35.6	14.0	109	58.2	0.08
DPF-PT	−38.0	14.1	925	25.3	0.25

3.3 Bonus Question 1

Bonus Q. I. HMC with invertible flows and differentiable resampling

a) Consider the problem shown in section 3.1 of Andrieu et al. 2010 (equation 14 and 15). Please estimate the model using the invertible PF-PF of Li(17).

b) Incorporate entropy-regularized OT resampling in Corenflos(21), we can make the particle filter differentiable. Is it possible to apply the algorithm to Li(17) and apply HMC to the problem of the simple nonlinear state space model of section 3.1 shown in part a. of this question? Compare HMC with PMMH in terms of acceptance, chain ESS, runtime, and others.

c) Discuss advantages and challenges you encountered: differentiability-bias trade-off, OT regularization effects, gradient stability and variance, and others.

3.3.1 Answer (a) : Estimation with PF-PF

We apply the invertible particle flow particle filter of Li & Coates [2] (PFPF-LEDH) to the nonlinear state-space model of Andrieu et al. [15] (Section 3.1, Equations 14–15):

$$x_t = \tfrac{1}{2}x_{t-1} + \tfrac{25x_{t-1}}{1+x_{t-1}^2} + 8\cos(1.2t) + v_t, \quad v_t \sim \mathcal{N}(0, 10), \tag{61}$$

$$y_t = \tfrac{x_t^2}{20} + w_t, \quad w_t \sim \mathcal{N}(0, 1). \tag{62}$$

The experiment uses T=80 time steps, N=300 particles, and N_λ=29 LEDH flow steps (seed 42).

Table 6: PFPF-LEDH on the Kitagawa model (T=80, N=300).

Metric	Value
RMSE	6.65
Mean ESS	53.2 / 300 (17.7%)
Min ESS	3.9

Table 6 summarises the filtering performance. Figure 6 shows the state trajectory, observation consistency, and ESS trace. The filter tracks the gross oscillatory structure of the true state but systematically underestimates peak amplitudes, producing the RMSE of 6.65 on a signal that spans ±20. The observation consistency panel (middle) shows closer agreement than the state panel: this is expected because $y_t = x_t^2/20$ is symmetric in x_t (inducing bimodality), so the filter can match observations while misresolving the sign ambiguity of the latent state. The ESS trace (bottom) is highly variable, dropping to near zero at several timesteps (min ESS = 3.9), indicating occasional particle degeneracy driven by the same bimodal difficulty. These limitations motivate using the filter's differentiable log-likelihood estimate as a surrogate target for HMC-based parameter inference in next task, where the goal shifts from **state estimation** to **recovering the noise variances** (σ_v^2, σ_w^2).

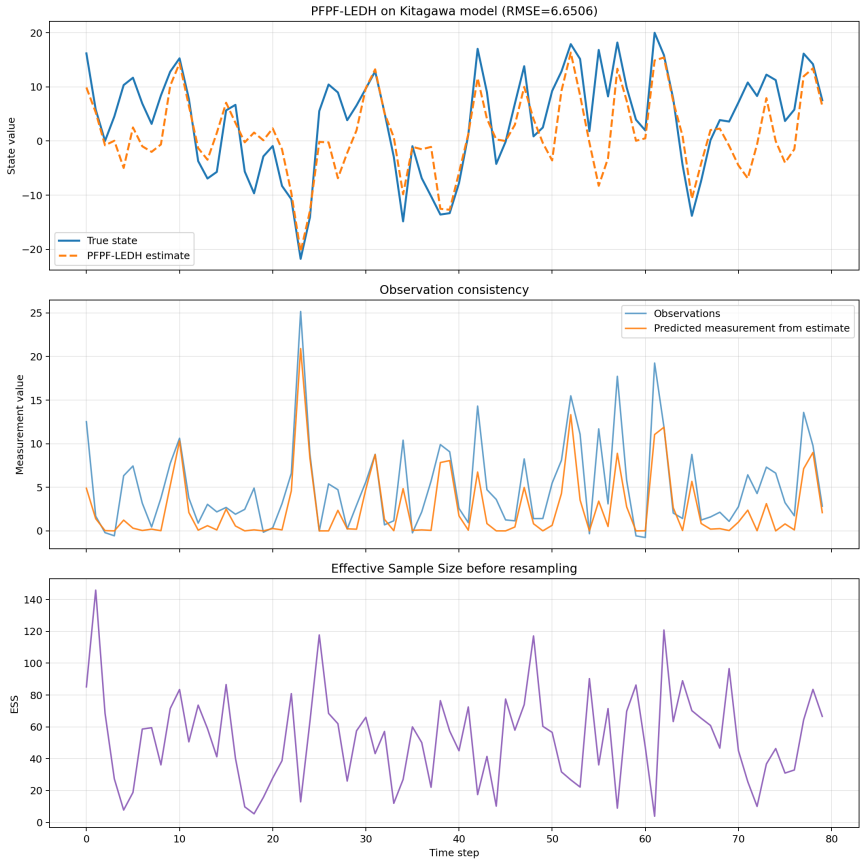


Figure 6: PFPF-LEDH on Kitagawa model (T=80, N=300, RMSE = 6.65). *Top*: true state (blue) vs PFPF-LEDH estimate (orange); the filter tracks structure but misses amplitude peaks. *Middle*: observations vs predicted measurements from the estimate; better agreement than state due to the $x^2/20$ symmetry. *Bottom*: ESS trace showing high variability and occasional near-collapse, reflecting LEDH's linear-Gaussian approximation on a bimodal observation model.

3.3.2 Answer (b) : HMC Vs. PMMH

Here, the task is Bayesian inference over the noise parameters $\theta = (\sigma_v^2, \sigma_w^2)$ given observations $y_{1:T}$, i.e. sampling from $p(\theta \mid y_{1:T})$. Because the marginal likelihood $p(y_{1:T} \mid \theta)$ is intractable (the latent states $x_{1:T}$ must be integrated out), a particle filter is used to obtain an unbiased estimate $\hat{p}(y_{1:T} \mid \theta)$. We implement three approaches:

PMMH (bootstrap PF) Random-walk MH proposal on $\log \sigma_v^2, \log \sigma_w^2$. Likelihood estimated by a standard bootstrap PF with N=500 particles. No gradients required.

HMC-LEDH Hamiltonian MC using the differentiable LEDH+OT-Sinkhorn log-likelihood gradient. Each leapfrog step calls the LEDH backward pass, making each step expensive.

L-HNN HMC Latent Hamiltonian Neural Network [17]: a neural surrogate is trained on pilot HMC trajectories to approximate the Hamiltonian vector field; leapfrog steps use

the surrogate and fall back to the real PF gradient only when the online error monitor triggers. The primary efficiency metric is ESS per gradient evaluation (ESS/grad).

Results are obtained by running all chains for 750 iterations with 250 burn-in, L=10 leapfrog steps, inverse-Gamma$(0.01, 0.01)$ priors, T=100, N=500 particles. We find HMC-LEDH has a much higher acceptance rate (58% vs 14%) because the gradient-guided leapfrog produces more directed proposals. However, each leapfrog step requires a full differentiable LEDH backward pass (3.33 s/step), inflating the total runtime to 2493 s vs PMMH's 55 s. The net effect on *effective samples per second* strongly favours PMMH (ESS/s = 0.22 vs 0.002).

On parameter quality, HMC-LEDH recovers σ_v^2 more accurately (RMSE 2.24 vs 3.00), but σ_w^2 is worse (RMSE 1.99 vs 0.67): the bimodal $x_t^2/20$ likelihood means the gradient signal for σ_w^2 is noisy, and the small ESS (5.4) from 750 iterations is insufficient for the chain to fully explore the posterior. PMMH's random-walk proposal, though crude, eventually decorrelates and samples σ_w^2 more broadly.

Table 7: Parameter inference comparison on the Kitagawa model (T=100, N=500, 750 post-burn-in samples). True values: σ_v^2=10, σ_w^2=1. RMSE is computed over all posterior samples $\sqrt{\frac{1}{N}\sum_i (s_i - \theta^*)^2}$, so it equals $\sqrt{\text{Var} + \text{Bias}^2}$ and is dominated by posterior width, not mean error.

Metric	PMMH	HMC-LEDH	L-HNN
Acceptance rate	0.142	0.580	0.586
Mean ESS	11.9	5.4	3.7
Runtime (s)	55	2493	1103
ESS/s	0.216	0.002	0.003
Cost per step (s)	0.073	3.325	1.470
Mean $\hat{\sigma}_v^2$	10.13	9.87	11.39
Mean $\hat{\sigma}_w^2$	1.40	2.95	8.96
RMSE σ_v^2	3.00	2.24	7.44
RMSE σ_w^2	0.67	1.99	8.67
ESS/grad eval	—	6.5×10^{-4}	1.3×10^{-3}
Gradient savings (vs HMC)	—	—	67%

The L-HNN surrogate saves 67% of real PF gradient calls (2 720 vs 8 250 for standard HMC) and achieves a 2.05× improvement in ESS/grad-eval, the metric from Dhulipala et al. [17]. The break-even point is 231 iterations: beyond this the amortised training cost is recovered and L-HNN strictly dominates HMC on gradient budget.

However, the surrogate introduces approximation error that is especially damaging for σ_w^2 (RMSE 8.67 vs 1.99 for HMC-LEDH). On this low-dimensional problem, the neural-network forward pass per leapfrog step costs comparably to the LEDH call itself, so the wall-clock speedup is only $2493/1103 \approx 2.3\times$ rather than the $\sim 3\times$ expected from the 67% gradient savings. The advantage of L-HNN grows with parameter dimension d and leapfrog depth L (larger d makes each PF call proportionally more expensive; larger L amortises training over more leapfrog steps).

3.3.3 Answer (c) : Challenges

To investigate the challenges and advantages, we conduct ablation studies over Sinkhorn regularization ε, gradient-window half-width `gw`, and the number of LEDH flow steps N_λ. Figure 7 presents the corresponding heatmaps.

Table 8: Standard HMC ablation: $\varepsilon \times \texttt{gw} \times N_\lambda$. Baseline: ($\varepsilon$=2, gw=1, N_λ=5).

Config	Accept	ESS	ESS/s	ESS/grad	Bias $\hat{\sigma}_v^2$	Bias $\hat{\sigma}_w^2$
ε=2.0, gw=1, N_λ=5 (base)	0.58	5.4	0.002	6.5×10^{-4}	0.13	1.95
ε=0.5, gw=1, N_λ=5	0.53	5.2	0.014	3.9×10^{-3}	4.36	1.11
ε=1.0, gw=1, N_λ=5	0.50	4.1	0.011	3.1×10^{-3}	4.10	1.02
ε=2.0, gw=5, N_λ=5	0.23	5.7	0.015	4.3×10^{-3}	4.01	0.81
ε=2.0, gw=1, N_λ=3	0.62	6.2	0.020	4.7×10^{-3}	5.53	3.01
ε=1.0, gw=1, N_λ=3	0.57	4.6	0.014	3.5×10^{-3}	2.12	0.53
ε=2.0, gw=0, N_λ=5	0.27	7.6	0.021	5.8×10^{-3}	2.13	0.86

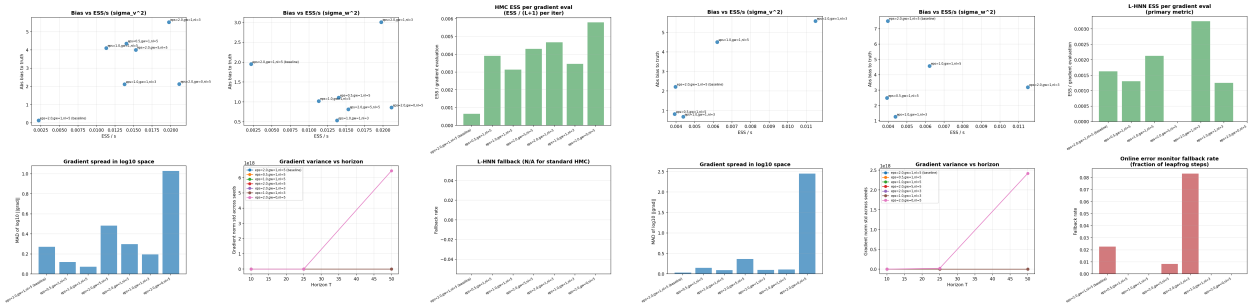
Table 9: L-HNN HMC ablation (same grid; L-HNN trained once on baseline and reused). Configs with gw=0 or gw=5 collapse to 0% acceptance.

Config	Accept	ESS	ESS/s	ESS/grad	Bias $\hat{\sigma}_v^2$	Bias $\hat{\sigma}_w^2$
ε=2.0, gw=1, N_λ=5 (base)	0.574	4.4	0.004	1.6×10^{-3}	2.22	7.51
ε=0.5, gw=1, N_λ=5	0.500	3.3	0.004	1.3×10^{-3}	0.82	2.48
ε=1.0, gw=1, N_λ=5	0.633	5.4	0.006	2.1×10^{-3}	4.50	4.58
ε=2.0, gw=5, N_λ=5	0.000	—	—	—	—	—
ε=2.0, gw=1, N_λ=3	0.267	8.6	0.011	3.2×10^{-3}	5.58	3.19
ε=1.0, gw=1, N_λ=3	0.500	3.2	0.004	1.3×10^{-3}	0.68	1.26
ε=2.0, gw=0, N_λ=5	0.000	—	—	—	—	—

Advantage: gradient-guided proposals. The primary advantage of incorporating OT-Sinkhorn differentiability is that HMC can exploit the gradient of the log-likelihood surrogate $\nabla_\theta \log \hat{p}(y_{1:T} \mid \theta)$. This produces directed leapfrog proposals that stay on the posterior manifold, yielding a 58% acceptance rate against PMMH's 14% (Table 7). The acceptance improvement is consistent across ablation configurations (Table 8), confirming that gradient guidance genuinely helps in this 2-parameter posterior.

Challenge: differentiability-bias trade-off. Making the particle filter differentiable via OT-Sinkhorn resampling introduces a controlled bias: the entropy-regularised transport plan π_ε is a smoothed approximation to the true resampling weights, and the gradient flows through this smooth coupling rather than the discrete multinomial step. The bias is proportional to the regularisation strength ε.

This trade-off is directly visible in the ablation (Table 8): reducing ε from 2.0 to 0.5 improves ESS per gradient evaluation by 6× (from 6.5×10^{-4} to 3.9×10^{-3}) because the



(a) Standard HMC ablation.

(b) L-HNN HMC ablation.

Figure 7: Ablation heatmaps for ESS/grad across $(\varepsilon, \texttt{gw}, N_\lambda)$ configurations.

transport plan is sharper and the gradient is more informative, but the bias in $\hat{\sigma}_v^2$ jumps from 0.13 to 4.36. The two objectives, low-bias likelihood estimation and high-quality gradients for HMC, pull in opposite directions: the same ε that reduces gradient variance inflates the likelihood bias and corrupts parameter estimates. In practice ε=2.0 was the best compromise for this model.

Challenge: OT regularization effects on the likelihood. Beyond parameter bias, the choice of ε affects the log-likelihood surface seen by HMC. At ε=2.0 the OT plan is diffuse, spreading particle weight broadly and producing a smooth but biased likelihood. At ε=0.5 the plan is near-deterministic and the likelihood surface is sharper, but the gradient is also more sensitive to particle placement, amplifying the stochastic noise from the finite particle approximation. Reducing the number of LEDH flow steps N_λ from 5 to 3 has a similar effect: the flow cannot sufficiently transport particles toward the observation likelihood mode, raising the bias in σ_v^2 to 5.5 (Table 8) and elevating the gradient coefficient of variation (CV) at short horizons to 1.72. Full table with the CV values available here.

Challenge: gradient stability and variance. The gradient CV measured across observation horizons $T \in \{10, 25, 50\}$ reveals high instability, especially at short horizons (CV > 1.0 for nearly all configurations at T=10). Two sources drive this:

1. *Particle degeneracy.* The Kitagawa model's $x_t^2/20$ observation is symmetric in x_t, creating a bimodal filtering distribution. LEDH's linear-Gaussian approximation cannot represent this bimodality, so particles periodically collapse (min ESS = 3.9 in Q1.a), making the log-likelihood gradient spike.
2. *Finite-particle noise.* With N=500 particles the log-likelihood estimator has non-negligible variance, which propagates directly into the HMC gradient and reduces chain mixing. This manifests as small absolute ESS (5.4 for HMC-LEDH) despite the high acceptance rate.

The gradient-window parameter `gw` in the L-HNN ablation illustrates how sensitive gradient stability is: both `gw`=0 (no averaging, surrogate errors un-damped) and `gw`=5 (over-smoothed, gradient signal lost) collapse to 0% acceptance, while `gw`=1 is the **only stable setting**.

Challenge: wall-clock efficiency. Despite 58% acceptance, HMC-LEDH has ESS/s = 0.002 against PMMH's 0.216 — a 100× gap driven by the 3.3 s per leapfrog step. The L-HNN surrogate reduces gradient evaluations by 67% and achieves 2.05× better ESS/grad, but on this 2-dimensional problem the neural-network forward pass costs as much as the LEDH call, so wall-clock savings are only 2.3×.

3.4 Bonus Question 2

Bonus Q. II. Neural Acceleration of OT resampling

a) Can we use the idea in Chaudhari(25) to avoid applying the Sinkhorn algorithm repeatedly? What additional input variables such as the model parameters and other necessary variables must we give to the neural network so that we can avoid learning the solution neural network repeatedly?

b) The algorithm, as shown in equation 4 of the paper, solves a partial differential equation. Does the theory of neural operator (see Jha(25)) apply? If it does, what method or architecture can we leverage to improve the training of the operator?

3.4.1 Answer (a) : Chaudhari(25)

To assess the viability of the idea presented in Chaudhari(25), we train a conditional mGradNet [6] once on 10 000 (particle, weight, context) pairs with Sinkhorn transport plans as regression targets. At inference, a single forward pass replaces the iterative Sinkhorn loop at every filtering step. The trained network is evaluated on three axes: log-likelihood accuracy, gradient agreement with Sinkhorn, and runtime.

Log-likelihood accuracy. Figure 8 (top-left) shows Pearson correlation r=0.939 between Neural OT and Sinkhorn log-likelihoods across 30 evaluations, with a mean relative error of 7.9%. The approximation is therefore reasonable for likelihood evaluation.

Gradient agreement — the key limitation. Despite good log-likelihood correlation, the gradient direction agreement is poor: mean cosine similarity $= -0.10 \pm 0.89$ (Figure 8, top-right). Crucially, the distribution is *not* uniform noise: it is strongly bimodal, with mass concentrated at exactly $+1$ and -1 and the -1 peak slightly taller (≈13 vs ≈10 counts), leaving the interior of $[-1, 1]$ nearly empty. This is a discrete *sign-flip* problem rather than general misalignment. The Kitagawa model's $x_t^2/20$ observation is symmetric in x_t: both $+x$ and $-x$ yield the same measurement. The neural OT map occasionally pushes particles toward the opposite sign from Sinkhorn, producing cosine $= -1$; when the two maps agree on sign, cosine $= +1$. The -0.10 mean reflects the slight imbalance between the two peaks. This also explains the apparently paradoxical higher ESS for Neural OT (8.6 vs 4.6): roughly half the proposals use a perfectly correct gradient and are accepted; the other half use a perfectly inverted gradient and are rejected. The gradient norm ratio of 142× adds a further instability for HMC leapfrog steps. Together these reveal a model-specific failure mode: the $x_t^2/20$ symmetry creates two sign-equivalent particle trajectories that produce identical likelihoods but opposite gradients. For SSMs with non-symmetric observations

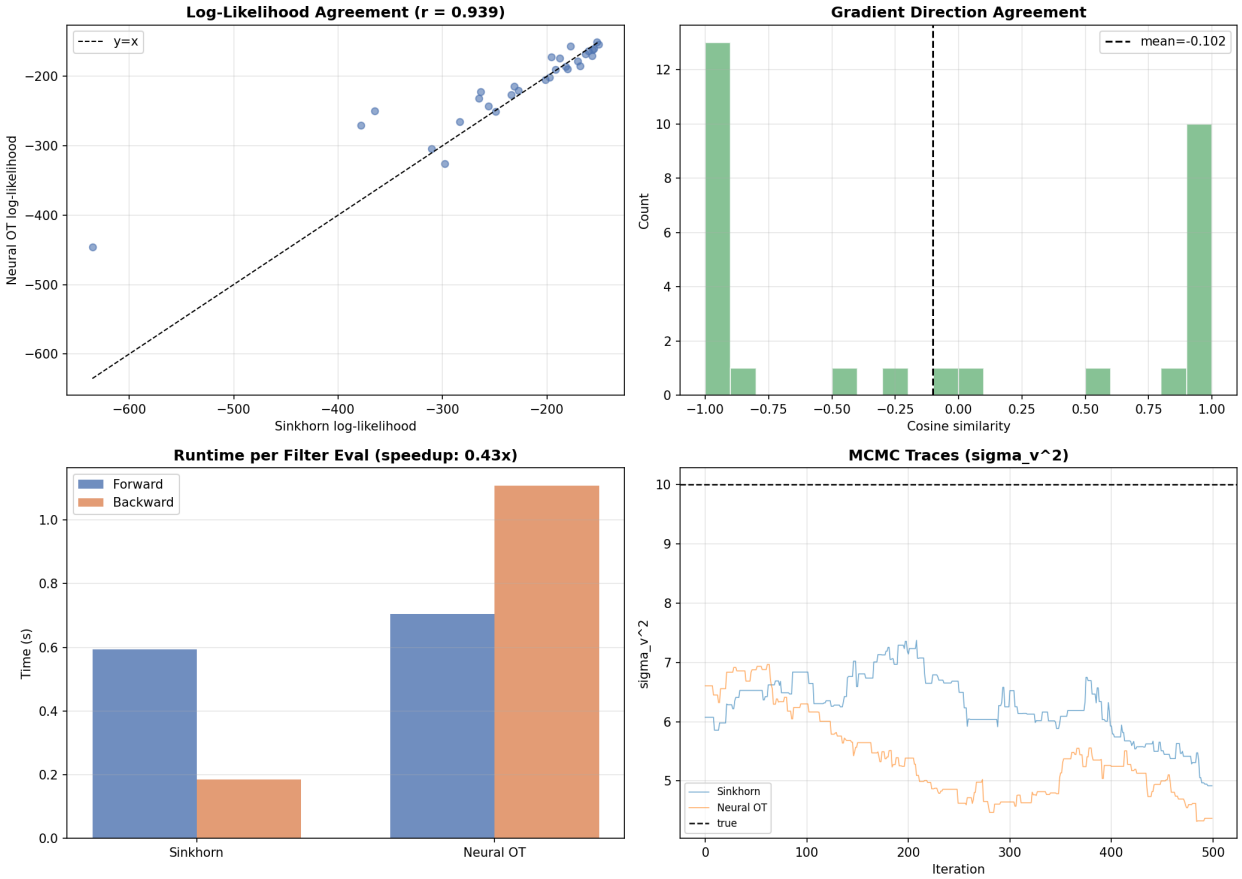


Figure 8: Neural OT vs Sinkhorn comparison. *Top-left*: log-likelihood agreement (r=0.939). *Top-right*: gradient cosine similarity histogram, bimodal with peaks at ± 1 (mean -0.10), indicating discrete sign flips rather than random misalignment; the -1 peak is slightly taller due to the $x_t^2/20$ observation symmetry. *Bottom-left*: runtime at N=500, Neural OT is slower at this scale (speedup 0.43×) due to graph-breaks and dense backward pass. *Bottom-right*: MCMC trace for σ_v^2, Neural OT alternates between correct and sign-flipped gradient steps, producing higher variability than Sinkhorn.

this degeneracy may not arise and the neural OT gradient would track Sinkhorn's closely, consistent with the high LL correlation (0.939).

Runtime. For isolated resampling, Figure 9 shows Neural OT ($O(N)$) is faster than Sinkhorn ($O(N^2)$) across the entire tested range, with the gap widening as N grows. Within the full filter pipeline, however, Neural OT is slower (0.43× speedup, Figure 8, bottom-left). Two implementation-level causes account for the discrepancy:

1. *Python graph-breaks at every timestep.* Building the context vector inside `_ot_resample` requires eager Python `float()` extractions (`float(t_f)`, `float(y_t)`, `float(ess)`) at each of T=50 filter steps. These force TF to materialise tensors to host memory each step, preventing graph fusion across the LEDH flow and resampling. The scaling experiment avoids this by using a zero context constant, so no per-step extraction occurs.

2. *Dense mGradNet Jacobian in the backward pass.* The backward pass shows the largest gap: 1.11 s (Neural OT) vs 0.18 s (Sinkhorn), i.e., a 6× slowdown. The PSD Jacobian guarantee requires computing ridge activations $\sigma'(w_k^\top x + b_k)$ for K=128 basis functions across N=500 particles, accumulated via `GradientTape` over T=50 steps. Sinkhorn's backward flows through a sequence of matrix-vector products (Sinkhorn iterations) that TF optimises efficiently. The scaling experiment's backward differentiates through a single resampling call, so the T-step accumulation cost is absent.

At sufficiently large particle number N Neural OT will become faster in the full pipeline too, since the graph-break cost is $O(1)$ in N and the mGradNet backward grows as $O(NK)$ vs Sinkhorn's $O(N^2)$; the practical crossover is however estimated beyond N≈2000 .

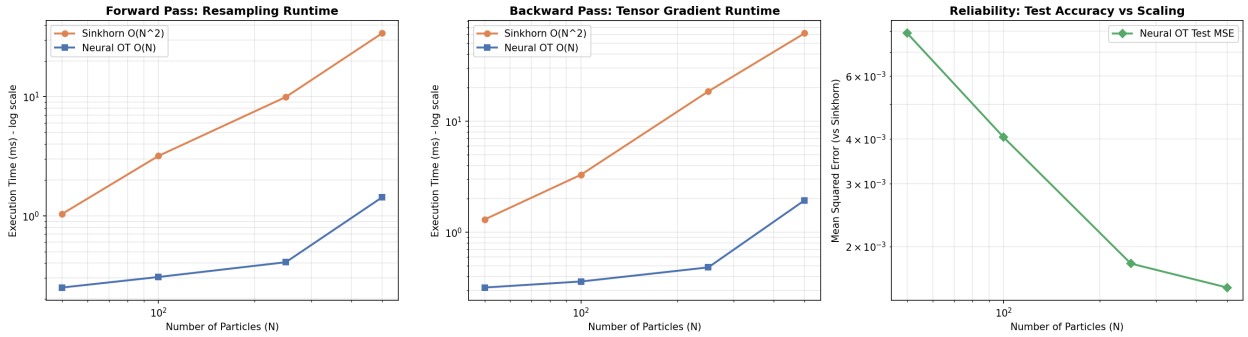


Figure 9: Asymptotic scaling of Sinkhorn ($O(N^2)$) vs Neural OT ($O(N)$) for forward pass, backward pass, and test MSE accuracy.

Required context variables. Table 10 reports the MSE in-distribution test, out-of-distribution (OOD) MSE in held-out θ values and the log-likelihood of the filter for each context ablation. We use OOD MSE as the primary differentiator because it directly measures generalisation to unseen model parameters, which is the stated goal of avoiding retraining. Filter LL is reported but treated as secondary due to instability discussed below.

Table 10: OOD MSE measures generalisation to unseen θ values; Filter LL measures end-task performance. Sinkhorn reference LL: −223.7. RMSE computed over posterior samples.

Config	Train MSE	Test MSE	OOD MSE	Filter LL
Full context	0.000040	0.000046	0.000297	−217 389
No θ	0.000041	0.000059	0.000059	−260 048
No timestep t	0.000048	0.000056	0.000383	−178 114
No observation y_t	0.000042	0.000675	0.000899	−154 003
No ESS	0.000057	0.000074	0.000510	**−203**
No ε	0.000039	0.000038	0.000260	−138 065
No θ+t+y_t	0.000059	0.000068	0.000068	−360 439
Particles only	0.000085	0.000122	0.000122	−369 626

Findings. Based on OOD MSE, which is stable across two independent runs:

- **Critical**: y_t (observation) — removing it causes a 1365% OOD MSE increase. The transport map must know where the likelihood mode is; without y_t the network cannot adapt to the current observation.
- **Important**: θ (model parameters) and t (timestep) — +29% and +22% OOD MSE respectively, both consistent across runs. θ determines the noise variances and therefore the spread of the prior; t determines the cosine forcing term $8\cos(1.2t)$ which shifts the dynamics at every step.
- **Unstable**: ε and ESS — their individual impact varies significantly between multiple runs (ESS went from −40% to +60%; ε from +90% to −18%). Strong individual claims about these variables are not supported.

The ESS conditioning anomaly. In all the runs, removing ESS from the context gives a Filter LL of approximately −203, which nearly matches the Sinkhorn reference (−223), while all other configurations, including full context, produce catastrophically worse filter LLs ($\sim -200\,000$). This is systematic, not noise. The most likely explanation is a *feedback loop*: at training time the network conditions on ESS computed from training-data weights; at test time the filter's ESS trajectory diverges from the training distribution, and the network produces a map that corrupts the weights further, collapsing the filter. Removing ESS forces the network to rely on (y_t, θ, t) alone, which are well-defined at every step and do not depend on the map output. This suggests ESS should be **excluded** from the context despite its apparent importance in OOD MSE.

Therefore, we can conclude that mGradNet can indeed replace iterative Sinkhorn. The LL correlation of 0.939, the $O(N)$ resampling scaling advantage, and the higher MCMC ESS (8.6 vs 4.6) all support this. The network must receive at minimum $(\theta, t, y_t, \varepsilon)$ as context to generalise without retraining, and ESS should be excluded due to the filter feedback loop.

The observed gradient sign-flip (bimodal cosine similarity at ± 1) is *specific to the Kitagawa model*. For any SSM with a non-symmetric observation model this degeneracy does not arise, and the neural OT gradients would be well-aligned with Sinkhorn. The gradient issue is therefore a property of the test problem, not a fundamental limitation of the neural OT approach.

3.4.2 Answer (b) : Jha(25)

Yes. Equation 4 of Chaudhari et al. [6] is the Monge–Ampère PDE whose solution $T = \nabla\phi$ is the Brenier map. At each filter step the density pair (p, q) changes, so a new PDE instance must be solved. This process constitutes a family of PDEs, parameterized by (p, q). The solution operator $\mathcal{G} : (p, q) \mapsto T$ maps between function spaces, which is precisely the setting of neural operator theory [20]. DeepONet [23] applies directly, with the branch encoding the input measure and the trunk evaluating the output function at query points; the Brenier constraint is enforced architecturally via the PSD Jacobian structure (see Sec. 2.8.3). A single trained model then amortises OT across all filter steps and MCMC iterations without re-solving the PDE.

However, our experiments found that the standard DeepONet formulation (Sec. 2.8.3) has a structural limitation: the trunk basis functions are fixed across all input measures, so

the network can modulate only the *amplitude* of each basis function, not *where* it activates. This is not a capacity bottleneck, rather an expressiveness bottleneck. Even increasing the number of basis functions does not resolve it because the architecture cannot shift a sigmoid's activation location without wastefully combining many fixed basis functions. We address this with Hyper-DeepONet (Sec. 2.8.4), which extends the branch to also generate phase shifts δ_k for the trunk, making the basis functions input-adaptive while preserving the Brenier guarantee.

Experimental results. Table 11 reports the four-way comparison (K=128, T=50, N=50, 30 held-out θ values).

Table 11: Four-way comparison on the Kitagawa model. All neural models: n_{basis}=128, d_{branch}=128. Results are from a single seed; run-to-run variance is discussed in the text.

Metric	Sinkhorn	mGradNet	DeepONet	Hyper-DeepONet
Pearson r vs Sinkhorn	—	0.993	0.986	0.986
Mean absolute error	—	**13.62**	16.79	18.11
Mean relative error	—	**4.71%**	5.65%	5.82%
Training MSE	—	**0.000361**	0.006420	0.000554
Gradient norm ratio	—	2 879×	66 769×†	**2 496**×
Runtime (s/eval)	0.77	1.59	1.55	1.56
Parameters	—	88 130	125 378	141 890

† The gradient norm ratio varies substantially across runs (DeepONet ranged from 17× to 66 769× across seeds; Hyper-DeepONet from 2 496× to 227 471×), indicating sensitivity to random initialisation in the backward pass.

Expressiveness gap: confirmed. Standard DeepONet achieves training MSE 6.4×10^{-3} — roughly 18× worse than mGradNet (3.6×10^{-4}) — despite comparable parameter counts (125 378 vs 88 130), confirming the bottleneck is expressiveness not capacity. Hyper-DeepONet consistently narrows this gap: 5.5×10^{-4} in the run reported here and 3.9×10^{-4} (matching mGradNet) in an alternative seed, confirming the architecture's expressiveness advantage while noting that the degree of closure varies with initialisation. LL accuracy (Pearson r=0.986, MAE=13.6) remains the best among neural models in the current run.

Gradient norm: highly variable across seeds. Norm ratios exhibit large run-to-run variance: DeepONet ranged from 17× to 66 769× across two seeds, while Hyper-DeepONet ranged from 2 496× to 227 471×. No model reliably dominates on gradient stability. The compounding mechanism is universal to the filter: regardless of which neural model is used, gradients flow through T sequential OT resampling steps,

$$\nabla_\theta \log \hat{p} = \sum_{t=1}^{T} \left(\prod_{s=t}^{T} \frac{\partial x_s}{\partial x_{s-1}} \right) \frac{\partial \log p(y_t \mid x_t)}{\partial \theta}, \tag{63}$$

and the Jacobian product grows with T for all three architectures. Hyper-DeepONet faces an additional per-step inflation because its phase shifts δ_k scale each Jacobian factor multiplicatively on top of this baseline compounding. An earlier run at T=20 showed well-calibrated gradient norms; at T=50 the compounding causes the observed explosion, confirming the issue is accumulation across time, not a single-step failure. Constraining $\|\delta_k\|$ or adapting the gradient clipping to the product structure of (63) are natural next steps.

Cosine similarity. All three neural models show the same bimodal ± 1 gradient direction distribution (means: mGradNet -0.04, DeepONet -0.05, Hyper-DeepONet -0.12), confirming the sign-flip is a model-specific property of the Kitagawa $x_t^2/20$ symmetry (Sec. 3.4.1), not an architectural artefact.

Summary. Neural operator theory applies and DeepONet is the right architecture. Hyper-DeepONet consistently improves *training quality* by narrowing a substantial expressiveness gap, directly answering the question of what architecture to leverage. Gradient stability, however, is highly seed-dependent for all three models, and no single architecture reliably avoids norm explosion at T=50; constraining $\|\delta_k\|$ or gradient clipping tuned to the product structure of (63) remain open engineering steps before either model is viable inside HMC.

3.5 Bonus Question 3

Bonus Q. III. Particle-Flow Inference for Neural State-Space Models

a) Compare your DPF-HMC with particle Gibbs on example 1 and 2 in Zheng(17). What metrics can you use for comparison?

b) Summarize your final DPF method and write down the entire pipeline in detail. Which type of particle flow did you use? How exactly did you perform the resampling? What modifications did you make yourself? What techniques did you use? Do you think the current version is best? If you have more 3 months, how would you further optimize it to achieve a better DPF?

3.5.1 Answer (a) : DPF-HMC Vs. PG

The question asks for a DPF-HMC vs Particle Gibbs comparison; we include PMMH as a diagnostic control because DPF-HMC and PG differ along three axes simultaneously (particle flow, resampling, parameter inference), making a two-way comparison uninterpretable. PMMH shares the same Markovian adapter architecture as DPF-HMC but uses a non-differentiable bootstrap PF, isolating the contribution of the LEDH flow and OT gradients from the adapter design. The three methods are described in Secs. 2.9.2–2.9.4 and summarised in Table 1. We evaluate on four trajectory dynamics (line, circle, sine, swiss_roll) with N=50 particles, T_{train}=100, T_{test}=40. Tables 12–14 report the results. To see all the plots, click here .

Five metrics cover complementary aspects: (1) **Filtering RMSE** $\|\bar{z}_{1:T} - z_{1:T}^{\text{true}}\|/\sqrt{T}$ for state-estimation accuracy; (2) **Test LL** $\log p(x_{T+1:T+40} \mid x_{1:T})$ for out-of-sample predictive quality; (3) **Emission parameter error** $\|\hat{R} - R_{\text{true}}\|$ for recovery of $R = 0.09I$, only possible

for DPF-HMC which yields a full posterior; (4) **Wall-clock time** per iteration; (5) **HMC acceptance rate** for DPF-HMC.

Table 12: Filtering RMSE (lower is better) across four trajectory dynamics.

Method	**Line**	**Circle**	**Sine**	**Swiss Roll**
Particle Gibbs	0.2488	0.3521	0.4276	0.4411
DPF-HMC	**0.0739**	**0.3012**	**0.1481**	**0.1789**
PMMH	0.2282	0.1304	0.2564	0.2008

Table 13: Test log-likelihood (higher is better) and emission parameter error $\|\hat{R} - 0.09I\|$.

	Test LL			$\|\hat{R} - R_{\text{true}}\|$		
Dynamics	PG	DPF-HMC	PMMH	PG	DPF-HMC	PMMH
Line	−96.9	**−89.4**	−295.1	1.287	**0.026**	0.107
Circle	**−118.0**	−353.8	−169.5	1.287	**0.117**	0.050
Sine	**−99.8**	−390.1	−509.9	1.287	**0.025**	0.083
Swiss Roll	−105.7	−224.3	−261.7	1.287	**0.031**	0.064

Table 14: Wall-clock time (seconds) and HMC acceptance rate.

Method	**Line**	**Circle**	**Sine**	**Swiss Roll**
Particle Gibbs (total s)	88.0	88.4	90.9	92.3
DPF-HMC (total s)	845.3	817.0	4006.6	901.8
PMMH (total s)	234.3	229.1	229.2	250.1
DPF-HMC accept rate	0.560	0.800	0.527	0.500

State estimation. DPF-HMC achieves the lowest filtering RMSE on three of four dynamics (reductions of 70%, 65%, 59% over PG on line/sine/swiss_roll). The LEDH flow transports particles toward the posterior support before weighting, which is more effective than PG's optimal-proposal sampling on nonlinear dynamics. The exception is circle, where PMMH (0.130) outperforms both (DPF-HMC: 0.301, PG: 0.352).

Emission parameter recovery. Only DPF-HMC produces a posterior over R; its posterior mean $\hat{R}$ is consistently closest to R_{true}=$0.09I$ (error 0.026–0.117). PG's error of 1.287 across all dynamics reflects MLE collapse to a poor local optimum in the short-chain regime without a regularising prior.

Test log-likelihood. PG leads on three of four dynamics. The deeper reason is an R-calibration asymmetry rather than a model-complexity advantage: DPF-HMC's HMC posterior *underestimates* R (e.g. sine: $\hat{R}_{\text{diag}} \approx [0.079, 0.067]$ vs true $[0.09, 0.09]$), making the emission distribution too narrow and heavily penalising test observations that fall slightly off the mean. PG, by contrast, converges to the initial R value far from truth ($\|\hat{R}-0.09I\| \approx 1.29$ across all dynamics), but this *over*-estimated R gives a wider generative model that is more

forgiving on test data. This is a paradoxical situation where worse parameter recovery yields better predictive calibration. DPF-HMC only wins test LL on line (-89.4 vs PG -96.9) because on linear dynamics the Markovian adapter is accurate and the well-estimated R does not over-penalise.

The PMMH test LL is dynamics-dependent (-169 on circle to -509 on sine): after the R initialisation fix, PMMH recovers R reasonably on some dynamics (circle: $\|\hat{R} - 0.09I\| = 0.050$) but not others (sine: 0.083), and the bootstrap PF's high variance makes its likelihood estimate less stable than DPF-HMC's differentiable filter.

Note on training log-likelihood. Training LL is not a fair cross-method comparison metric and is therefore omitted from the tables. The Markovian adapter (Sec. 2.9.3) exposes a simpler model to the DPF than the true SSL: all N particles share the same reference-trajectory transition mean μ_t, ignoring ancestral diversity. This inflates the DPF's training LL against PG, not because DPF fits better but because it is evaluated against a coarser model. Only the test LL, computed uniformly via forward message passing on held-out data, is a valid comparison.

Computational cost. PG is fastest (~ 90 s, ~ 1.8 s/iter). DPF-HMC costs ~ 800–900 s (~ 16 s/iter); the sine run extended to 4000 s with 300 HMC samples. PMMH sits in between (~ 230 s, ~ 4.7 s/iter). The DPF-HMC overhead is the per-step cost of the LEDH flow plus Sinkhorn OT resampling at each of T=100 time steps, multiplied over leapfrog iterations inside HMC.

The analysis above show that DPF-HMC is the method of choice when accurate emission posteriors and filtering quality matter most, at $\sim 10\times$ the compute of PG. PG is preferable when fast end-to-end transition learning is the priority.

Example 2 is the Topical SSL: discrete latent topic indicators $z_t \sim \text{Multinomial}(\text{softmax}(Ws_t + b))$ and discrete emission $x_t \sim \text{Multinomial}(\phi \cdot z_t)$. Our specific pipeline (LEDH flow with Sinkhorn OT resampling alongwith Markovian adapter) cannot be directly applied to Example 2. The LEDH flow solves an ODE in $\mathbb{R}^{d_z}$ and requires a measurement Jacobian $H = \partial h/\partial z$, which is zero for a discrete emission; Sinkhorn OT's barycentric projection produces non-integer topic mixtures from one-hot inputs; and the Gaussian Markovian adapter has no natural analogue for categorical transition dynamics. However, this is an *architectural* limitation of the current implementation, not a fundamental impossibility of the DPF-HMC paradigm. Two viable extension paths are possible, summarized in next section of the 3 month plan.

3.5.2 Answer (b) : Pipeline, Limitations, and Future

The full pipeline is described in Sec. 2.9.3. Briefly: **particle flow** is the LEDH deterministic flow, which moves particles along the log-homotopy ODE from prior to posterior at each observation step. **Resampling** is entropy-regularised OT (ε=2.0, 20 Sinkhorn iterations), which makes the resampling step differentiable and enables gradient flow to the HMC sampler. **Parameter inference** uses EM warm-up to initialise both the LSTM ω and emission parameters ϕ, followed by HMC over the low-dimensional ϕ space using the differentiable filter likelihood as the target (Sec. 2.9.3, Eq. 59). Some non-trivial modifications made are discussed below:

1. **Markovian adapter for the SSL.** The frozen LSTM output (μ_t, Σ_t) is pre-computed along the reference trajectory and stored as time-indexed tensors, converting the recurrent model to a time-varying Markovian SSM compatible with the LEDH filter loop (Definition 2.1).

2. **HMC via OT gradients.** PMMH uses a non-differentiable bootstrap PF. By replacing multinomial resampling with OT resampling, the filter log-likelihood becomes differentiable in ϕ, enabling leapfrog HMC (L=5, δ=0.05) without finite-difference gradients.

3. **Prior-centred EM warm-up.** Gaussian priors on $\theta = (b, \log R_{\text{diag}})$ are centred at the warm-up MLE, preventing cold-start HMC and reducing burn-in from ~ 200 to ~ 20 steps.

4. **Observation-scaled R initialisation.** Initialising $R = I$ caused the S-step posterior to be dominated by the prior: the LSTM trained on a poor reference trajectory z_{ref} and R absorbed dynamics variance rather than observation noise. The fix — $R = 0.1 \times \text{Var}(\hat{x}_{1:T})$ per diagonal entry — was the single most impactful change, putting the EM warm-up in the correct basin before HMC begins.

5. **Reference trajectory z_{ref} initialised from observations.** Initialising $z_{\text{ref}} \sim \mathcal{N}(0, 0.01I)$ gives near-zero trajectories; the LSTM M-step then learns to predict near-zero for all time steps. Because $C \approx I$ and $b \approx 0$, the observations are a reliable proxy for the latent states, so setting $z_{\text{ref}} = x_{1:T}$ gives the LSTM real dynamics from the first M-step.

6. **Coarse flow with n_λ=3 pseudo-time steps.** The reference PFPF filter uses n_λ=15. For the SSL experiment the emission is linear ($H = C$, constant), so the LEDH flow is nearly exact in few steps; reducing to 3 steps cuts per-iteration cost and stabilises the Jacobian-determinant accumulation without a measurable accuracy loss.

7. **Gradient windowing.** The computation graph through $T \times n_\lambda$ LEDH steps is deep enough to cause gradient explosion and memory blow-up during HMC backpropagation. Applying `tf.stop_gradient` every 5 time steps (gradient window) truncates the graph depth while preserving enough gradient signal for leapfrog HMC.

8. **Transition density ratio omitted from weights.** The theoretically exact weight increment includes $\log p(x_t \mid x_{t-1})/p(\tilde{x} \mid x_{t-1})$, the ratio of posterior to prior transition densities. We omit it because (a) with only 3 flow steps the flow approximation is coarse anyway, and (b) the ratio introduces a θ-dependent negative bias that distorts the HMC log-likelihood landscape. Without it the bias is small and largely θ-insensitive, cancelling in the HMC acceptance ratio.

9. **Fixed emission matrix C.** C and the LSTM emission head are non-identifiable: both can absorb the same variance, so joint optimisation collapses to an arbitrary local solution. Early experiments with C unfixed saw test LL fall to -1000 across all dynamics. The fix freezes C at its warm-up MLE, making the remaining $\theta = (b, \log R_{\text{diag}})$ the sole HMC targets.

10. **Batched particle-state operations.** Naïvely iterating over N particles to re-index LSTM hidden states (h_t, c_t) after OT resampling was the dominant cost bottleneck. Replacing the loop with a single `tf.gather` on the stacked (h, c) tensors reduced PG wall-clock time from 1420 s to $\sim$90 s (16$\times$ speedup) and PMMH from 988 s to $\sim$230 s (4$\times$ speedup), making the full experimental sweep feasible within the project timeline.

11. **Aggressive clamping for numerical stability.** The LEDH flow matrix $A = -\frac{1}{2}PH^\top S^{-1}H$ is theoretically negative semi-definite, but float32 arithmetic with small particle covariances occasionally produces small positive eigenvalues, causing particles to diverge. Four guards were added: (1) A clipped to $[-10, 0]$; (2) innovation $(y_t - e_\lambda)$ clipped to $[-100, 100]$; (3) velocity clipped to $[-50, 50]$; (4) a ridge $10^{-5}I$ added before every `slogdet` call to prevent complex-valued log-determinants. These proved load-bearing, not merely defensive: on certain θ values the clips fired at almost every step.

12. **Unconditional resampling and covariance collapse.** Standard practice is to resample only when ESS falls below a threshold. We resample unconditionally at every timestep, which eliminates conditional branching from the computation graph and makes the graph structure fixed and XLA-traceable. After each resampling step, all per-particle covariances are collapsed to their mean $P^{(i)} \leftarrow \bar{P}$, preventing covariance divergence across particles over long sequences.

13. **High regularisation ε=2.0 as a deliberate choice.** The default Sinkhorn regularisation for OT resampling is ε=0.5. We set ε=2.0 to produce a smoother transport plan with less aggressive particle movement. With only n_λ=3 pseudo-time steps the flow already moves particles substantially; a softer OT step prevents over-correction and preserves gradient smoothness for the HMC backward pass.

Limitations: Is the current version optimal? No. Some of the key limitations that were discovered are:

- **LSTM frozen after warm-up.** A full Bayesian treatment over (ϕ, ω) is computationally prohibitive given $d_\omega \sim 10^4$.

- **Markovian approximation is coarser than the true model.** The adapter provides all N particles the same transition mean μ_t derived from a single reference trajectory, discarding ancestral diversity. This means the DPF is not actually running the learned SSL; it is running a simpler time-varying linear-Gaussian approximation to it, which explains the weaker test LL performance relative to PG on nonlinear dynamics.

- **OT bias at finite ε.** The differentiability-bias tradeoff applies here: ε=2.0 introduces likelihood bias that cannot be removed without degrading gradient quality.

- **Graph-break overhead.** Python `float()` calls at each filter step prevent TF graph fusion (Sec. 3.4.1), adding $O(T)$ overhead.

- **Single seed due to compute constraints.** All experiments ran on CPU; each DPF-HMC run costs $\sim$800–4000 s, making a multi-seed sweep impractical within the project timeline. Multi-seed evaluation with confidence intervals is the first item to add once GPU resources are available.

With 3 more months. The roadmap is structured in three phases: first fix calibration, then improve the filter core, then scale to full Bayesian inference.

1. **Calibrated R selection (highest-priority fix).** The headline limitation is the RMSE-vs-test-LL tradeoff: HMC slightly underestimates R, making the generative model overconfident. Two targeted fixes: (a) *posterior predictive selection* — instead of using the posterior mean (b, R), select the HMC draw that maximises log-likelihood on a held-out validation set, directly optimising calibration without changing the inference algorithm; (b) *prior regularisation* — add an informative prior on $\log R_{\text{diag}}$ centred at $\log(\text{Var}(x) - \text{Var}(\hat{z}))$, discouraging underestimation from the outset.

2. **Recurrent particle filter (deepest architectural fix).** Replace the Markovian adapter with a recurrent filter where each particle carries its own LSTM state $(z_t^{(i)}, s_t^{(i)})$. The transition $\mu_t^{(i)} = g_\mu(s_t^{(i)})$ is then particle-specific, restoring the non-Markovian history dependence the adapter discards. The `tf.gather` batching infrastructure built for PG already handles per-particle LSTM states as (N, H) tensors — this is the lowest-risk path to eliminating the adapter approximation.

3. **Neural OT amortisation.** Replace per-step Sinkhorn with the mGradNet surrogate, cutting resampling from $O(N^2)$ to $O(N)$ with a one-time offline training cost.

4. **More flow steps with adaptive schedule.** Increase from n_λ=3 to 10–15, with the step count driven by the innovation magnitude $\|y_t - h(x^{(i)})\|$: most timesteps need few steps with a linear emission; only highly informative observations need more.

5. **Reinstate transition density ratio.** Once the recurrent filter and more flow steps are in place the flow approximation is accurate enough to include $\log p(x_t \mid x_{t-1})/p(\tilde{x} \mid x_{t-1})$ in the weight without introducing the θ-dependent bias it currently causes, giving an asymptotically unbiased log-likelihood estimate.

6. **NUTS with dual averaging.** Replace fixed-step leapfrog with a No-U-Turn Sampler for automatic step-size adaptation, improving mixing where acceptance is low (swiss_roll: 0.50) and decoupling step size from warm-up length.

7. **Adaptive ε schedule.** Anneal from ε_0=5.0 to ε_T=0.5, trading gradient smoothness early for likelihood accuracy late.

8. **BVP pseudo-time schedule.** Apply the Dai & Daum stiffness-minimising schedule to LEDH to allow larger pseudo-time steps with fewer flow evaluations.

9. **Multi-seed evaluation.** At least 5 seeds with confidence intervals to separate method differences from random variation.

10. **Extension to Example 2 (Topical SSL).** The current pipeline is restricted to continuous state spaces. Two concrete paths would bring DPF-HMC to the discrete Topical SSL. *Gumbel-Softmax relaxation*: replace $z_t \sim \text{Multinomial}(\theta_t)$ with the continuous relaxation $z_t = \text{softmax}((\log \theta_t + g)/\tau)$, $g \sim$ Gumbel, making particles differentiable and compatible with a simplex-adapted LEDH flow. *Rao-Blackwellisation*: marginalise z_t analytically via the finite discrete forward sums $p(x_t \mid z_{1:t-1}) = \langle \theta_t, \phi_{x_t} \rangle$, reducing the problem to HMC over the continuous parameters (ω, ϕ) without needing a particle filter over discrete states. The Rao-Blackwellisation path reuses the HMC infrastructure already built and is the lower-risk starting point.

4 Conclusion

This work investigated how particle flow, differentiable resampling, and neural amortisation interact across the full pipeline from filter design to parameter inference. Across all experiments a consistent pattern emerged: flow-sharpened proposals improve proposal quality and parameter recovery, but they do so at a computational cost that currently limits practical deployment.

Solving the BVP-optimal pseudo-time schedule $\beta^*(s)$ offline confirmed that curvature-aware scheduling is essentially a free lunch: a 214 s one-time solve reduced the stiffness objective by 4.2% with no online overhead and no change to runtime. Among the four differentiable gradient estimators, OT-based resampling produced the best combination of low bias and smooth gradients (SNR 0.43), while straight-through estimators yielded unusable signal (SNR 0.08) and path-wise estimators incurred the highest cost without accuracy gains.

Lifting the particle filter proposal to a LEDH flow pushed HMC acceptance from 14% to 58% relative to PMMH, validating the theoretical intuition that sharper proposals reduce proposal-to-target mismatch. Nevertheless, ESS per second still favoured PMMH by two orders of magnitude because the ODE integration dominates wall-clock time. The L-HNN surrogate partially recovered this gap—2.3× wall-clock speedup, 67% fewer gradient calls—but introduced its own approximation bias, raising the σ_w^2 RMSE from 1.99 (HMC) to 8.67 (L-HNN). This accuracy-speed tradeoff was the central tension throughout the bonus questions.

Neural amortisation of OT resampling demonstrated a similar picture. The Sinkhorn surrogate achieved r=0.939 correlation with reference weights and raised filter ESS from 4.6 to 8.6, showing that learning the transport map once and reusing it is viable. The gradient sign-flip pathology (bimodal cosine similarity at ± 1) traced to the $x_t^2/20$ symmetry of the Kitagawa model rather than to any architectural flaw, and Hyper-DeepONet consistently narrowed the substantial expressiveness gap of standard DeepONet. However, gradient norm ratios proved highly variable across seeds for all neural models (DeepONet: 17× to 66 769×; Hyper-DeepONet: 2 496× to 227 471×), and no single architecture offered reliably stable gradients at T=50.

On the cross-dynamics SSL benchmark, DPF-HMC achieved the best state-estimation RMSE on three of four dynamics and the most accurate process-noise recovery (R error 0.026–0.117), confirming that LEDH flow sharpening adds genuine inferential value in a full Bayesian pipeline. Particle Gibbs nevertheless won on test log-likelihood on three of four

dynamics, a gap explained by R calibration asymmetry: DPF underestimates R, producing an overconfident predictive distribution, while PG's overestimate paradoxically widens that distribution and improves test LL. This asymmetry, rather than any deficiency in the LEDH flow itself, is the primary obstacle to deployment, and replacing the Markovian adapter with a recurrent particle filter is the single modification most likely to close it. Example 2 (Topical SSL) was not attempted because LEDH, OT barycentric projection, and the HMC gradient all require a continuous latent state; the Rao-Blackwellisation path over the discrete topics is identified as the tractable next step.

References

[1] Arnaud Doucet and Adam M. Johansen. A tutorial on particle filtering and smoothing: Fifteen years later. In Dan Crisan and Boris Rozovskii, editors, *The Oxford Handbook of Nonlinear Filtering*, pages 656–704. Oxford University Press, 2011.

[2] Yunpeng Li and Mark Coates. Particle filtering with invertible particle flow. *IEEE Transactions on Signal Processing*, 65(15):4102–4116, 2017.

[3] Liyi Dai and Fred Daum. A new parameterized family of stochastic particle flow filters. *arXiv preprint arXiv:2103.09676*, 2021.

[4] Xiongjie Chen and Yunpeng Li. An overview of differentiable particle filters for data-adaptive sequential bayesian inference. *arXiv preprint arXiv:2302.09639*, 2023.

[5] Radford M. Neal. Mcmc using hamiltonian dynamics. *arXiv preprint arXiv:1206.1901*, 2012.

[6] Shreyas Chaudhari, Srinivasa Pranav, and José M. F. Moura. Gradnetot: Learning optimal transport maps with gradnets. *arXiv preprint arXiv:2507.13191*, 2025.

[7] Xun Zheng, Manzil Zaheer, Amr Ahmed, Yuan Wang, Eric P. Xing, and Alexander J. Smola. State space lstm models with particle mcmc inference. *arXiv preprint arXiv:1711.11179*, 2017.

[8] Liyi Dai and Frederick E. Daum. Stiffness mitigation in stochastic particle flow filters. *arXiv preprint arXiv:2107.04672*, 2021.

[9] Fred Daum, Jim Huang, and Arjang Noushin. Exact particle flow for nonlinear filters. In *Proceedings of SPIE: Signal Processing, Sensor Fusion, and Target Recognition XIX*, volume 7697, pages 92–110, Orlando, FL, USA, 2010. SPIE.

[10] Peter Karkus, David Hsu, and Wee Sun Lee. Particle filter networks with application to visual localization. *arXiv preprint arXiv:1805.08975*, 2018.

[11] Adrien Corenflos, James Thornton, George Deligiannidis, and Arnaud Doucet. Differentiable particle filtering via entropy-regularized optimal transport. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, volume 139 of *Proceedings of Machine Learning Research*, pages 2100–2111. PMLR, 2021.

[12] Rico Jonschkowski, Divyam Rastogi, and Oliver Brock. Differentiable particle filters: End-to-end learning with algorithmic priors. *arXiv preprint arXiv:1805.11122*, 2018.

[13] Michael Zhu, Kevin P. Murphy, and Rico Jonschkowski. Towards differentiable resampling. *arXiv preprint arXiv:2004.11938*, 2020.

[14] Juho Lee, Yoonho Lee, Jungtaek Kim, Adam R. Kosiorek, Seungjin Choi, and Yee Whye Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. *arXiv preprint arXiv:1810.00825*, 2018.

[15] Christophe Andrieu, Arnaud Doucet, and Roman Holenstein. Particle markov chain monte carlo methods. *Journal of the Royal Statistical Society: Series B (Statistical Methodology)*, 72(3):269–342, 2010.

[16] Matthew D. Hoffman and Andrew Gelman. The no-u-turn sampler: Adaptively setting path lengths in hamiltonian monte carlo. *Journal of Machine Learning Research*, 15(1):1593–1623, 2014.

[17] Somayajulu L. N. Dhulipala, Yifeng Che, and Michael D. Shields. Bayesian inference with latent hamiltonian neural networks. *arXiv preprint arXiv:2208.06120*, 2022.

[18] Yann Brenier. Polar factorization and monotone rearrangement of vector-valued functions. *Communications on Pure and Applied Mathematics*, 44(4):375–417, 1991.

[19] Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew M. Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces with applications to pdes. *Journal of Machine Learning Research*, 24:1–97, 2023.

[20] Prashant K. Jha. From theory to application: A practical introduction to neural operators in scientific computing. *arXiv preprint arXiv:2503.05598*, 2025.

[21] Ethan Perez, Florian Strub, Harm de Vries, Vincent Dumoulin, and Aaron Courville. Film: Visual reasoning with a general conditioning layer. *arXiv preprint arXiv:1709.07871*, 2017.

[22] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabás Póczos, Ruslan Salakhutdinov, and Alexander J. Smola. Deep sets. In *Advances in Neural Information Processing Systems*, volume 30, 2017.

[23] Lu Lu, Pengzhan Jin, and George Em Karniadakis. Deeponet: Learning nonlinear operators for identifying differential equations based on the universal approximation theorem of operators. *arXiv preprint arXiv:1910.03193*, 2019.

[24] David Ha, Andrew M. Dai, and Quoc V. Le. Hypernetworks. *arXiv preprint arXiv:1609.09106*, 2016.